Spectral–Spatial Classification of Few Shot Hyperspectral Image With Deep 3-D Convolutional Random Fourier Features Network

Tingting Wang, Huanyu Liu, and Junbao Li

***Abstract*— Remote sensing hyperspectral images are very useful for land cover classification because of their rich spatial and spectral information. However, hyperspectral image acquisition and pixel labeling are laborious and time-consuming, so few-shot learning methods are considered to solve this problem. Deep learning has gradually been used for few-shot hyperspectral classification, but there are some problems. The feature extraction network based on deep learning requires too many parameters to be trained, resulting in a huge network model, which is not conducive to deployment on remote sensing data acquisition equipment. Moreover, due to the lack of label samples, the algorithm based on deep learning is more prone to overfitting. To solve the above problems, considering the advanced characteristics of the kernel method in dealing with nonlinear, small sample, and high-dimensional data, we propose a small scale high precision network called 3-D convolution random Fourier features (3-DCRFF) based on the random Fourier feature (RFF) kernel approximation, which is the 3-DCRFF network. First, we combine 3-D convolution with RFF as the basic structure of the network to extract the spatial and spectral features of HSI cubes. Second, we use a classifier based on attention mechanism to classify feature vectors to obtain recognition probability. Finally, the network parameters are solved from the perspective of Bayesian optimization, and the synthetic gradient optimization method is designed and implemented to realize the fast learning of the network. A large number of experiments HSI classification experiments were performed on University of Pavia (UP), Pavia Center (PC), Indian Pines (IP), and Salinas standard remote sensing datasets, the results show that our algorithm outperforms most state-of-the-art algorithms on few-shot classification.**

***Index Terms*— 3-D convolution, deep kernel learning, few-shot learning, hyperspectral image classification, random Fourier features (RFF).**

I. Introduction

HYPERSPECTRAL images contain rich spatial and spectral information, which is an important means of Earth observation in the field of remote sensing. Therefore, hyperspectral image information processing and analysis techniques are also one of the key research contents in the field of remote sensing. For hyperspectral images, 2-D spatial information is as important as 1-D spectral information [1]. The main idea of this method is to make full use of spatial–spectral information in remote sensing image cube. Hyperspectral images are widely used in environmental monitoring [2], accurate agriculture [3], geological exploration [4]–[7], and vegetation ecology [8]. In the last few decades, many methods have been proposed for HSI classification [9]–[11].

As spectral resolution increases, it is worth considering processing data directly from satellites or drones. The computing resources on these devices are limited and a small-scale model is required. Moreover, two common conditions of hyperspectral application are considered.

1) The spectral varies greatly in different geographical environments. In order to ensure the classification effect, these samples need to be labeled and time-sensitive to meet the application of spectral classification under environmental changes. The time cost of sample labeling is very high, and it is not easy to obtain large labeled samples, which is a typical problem of few-shot.

2) In HSI data, the spectral of samples collected in the same geographical are different at different times. As the application conditions such as weather and temperature change, the classification model needs to be updated frequently. The new data is similar to the original data, but the amount of data is small. The few-shot learning method can effectively learn the new data.

Aiming at the above requirements of HSI classification, the research focus of this article is to design a small-scale network for few-shot data. The deep learning method has effect in the field of image processing, but it needs a large number of labeled samples to train the model. When the samples scarce, the deep learning network is prone to overfitting. In addition, the deep learning network has too many parameters and requires high computing resources. For example, high-performance GPU computing is required. Inspired by the above problems, we consider using traditional machine learning methods to achieve few-shot hyperspectral image classification. The kernel method performs well on small samples, nonlinear and high-dimensional data, but faces some challenges. Single kernel functions such as radial basis kernel (RBF) are limited by data expression ability, and the accuracy can not reach the optimal situation, and the memory consumption and computational complexity are as high as $O(n^2)$ and $O(n^3)$, respectively. In order to solve the above problems, we consider to deepen traditional methods such as kernel method and make full use of kernel method's

Manuscript received 13 March 2022; revised 2 May 2022 and 27 May 2022; accepted 19 June 2022. Date of publication 23 June 2022; date of current version 7 July 2022. This work was supported in part by the Interdisciplinary Research Foundation of HIT under Grant IR2021104 and in part by the Aeronautical Science Foundation of China under Grant 2019ZC077006. *(Corresponding author: Junbao Li.)*

The authors are with the School of Cyberspace Science, Harbin Institute of Technology, Harbin 150001, China (e-mail: junbaoli82678474@126.com).

Digital Object Identifier 10.1109/TGRS.2022.3185612

1558-0644 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:52:12 UTC from IEEE Xplore. Restrictions apply.

adaptability to small samples and strong feature expression ability of deep structure to build a novel lightweight few-shot learning network. The RFF kernel approximation [12] avoids the process of calculating the inverse of the matrix in the conventional kernel method, reduces the number of parameters and improves the efficiency of the model.

Based on the advantages of the kernel approximation method, we proposed the 3-D convolution random Fourier features (3-DCRFF) method in this article, which realized the classification of HSI with scarce data (five samples per class) and achieved a compromise between classification accuracy and efficiency. The main contributions of this article are as follows.

1) We propose a novel deep 3-DCRFF kernel approximation framework, and combine it with a cosine similarity-based classifier to achieve few-shot HSI classification.
2) We design a synthetic gradient network to ensure the back propagation of the model and solved the proposed model parameters from the perspective of Bayesian optimization.
3) The proposed 3-DCRFF method uses meta-learning to obtain HSI classification models, which effectively improves the environmental adaptability of model. We adopt two-stage training. The 3-DCRFF network learns the feature representation of the data, and then adopt the meta-learning strategy to training the classifier. This method not only ensures that the algorithm has high feature extraction ability but also has the ability of fast generalization to new data.

The 3-DCRFF model is constructed for the classification of few-shot hyperspectral images. Aiming at the problem of spectral changes of hyperspectral images at different time points, a 3-DCRFF model combined with few-shot learning strategy can quickly generalize to hyperspectral classification with insufficient samples. 3-DCRFF as a nonparametric Bayesian model has low requirements for computing equipment and storage space, it is suitable for hyperspectral applications. The 3-DCRFF theory is clear and suitable for the application of high requirements for algorithm interpretation, such as hyperspectral.

The remainder of this article is organized as follows. In Section II, the proposed 3-DCRFF framework is described in detail. In Section III, experimental settings and the datasets are provided. In Section IV, experimental results on four public available HSI datasets and the comparison results are presented. Finally, we make a summary of the research in this article.

II. Related Work

Much efforts have been devoted to solve the hyperspectral classification problem. In the following, we will introduce the methods related to our work from three aspects, including the traditional hyperspectral classification method, the method based on deep learning and the few shot learning method based on the scarcity of hyperspectral data.

A. Traditional Methods

The traditional machine learning methods, including support vector machines (SVM) [13], k-nearest neighbor (KNN) [14], random forest [15], have been studied for HSI classification. In the early studies, these traditional methods were regarded as mainstream technologies. However, since the classification performance of these algorithms was largely affected by the curse of dimensionality, it is necessary to combine with band selection [16], [17] and dimension reduction techniques including principal components analysis (PCA) [18], independent component analysis (ICA) [19].

Because of the shallow structure of the traditional machine learning method, the expression ability of the algorithm is limited, and it can not effectively deal with high-dimensional data. However, traditional methods can deal with nonlinear small sample problems and avoid overfitting effectively. Making full use of the adaptability to small samples of traditional methods such as kernel approximation and the powerful feature extraction ability of deep learning is the way to solve this problem.

B. Deep Learning Methods

Recently, deep learning has achieved great success in the field of image processing, including image classification [20]–[22], object detection [23]–[25], image segmentation [26]–[28] and other applications [29]. Deep learning is an inevitable trend in hyperspectral image classification [30]. A large number of research works have emerged in the field of hyperspectral image classification based on deep learning. Stacked autoencoder [31] is used for high-level feature extraction of HSI, which opens the door of deep learning for remote sensing image processing. Subsequently, convolutional neural networks (CNNs) [32], recurrent neural networks (RNNs) [33], and deep belief networks [9] with outstanding performance in deep learning are applied to HSI classification successively. Hyperspectral remote sensing image is an image cube composed of 2-d geometric spatial information and 1-d spectral information. Given the characteristics of hyperspectral images, researchers proposed the classification methods of 1-D-CNN [34] using only spectral information, 2-D-CNN [35]–[38] using spatial information, and 3-D-CNN [39]–[41] using spatial–spectral combination features. It has been proven that the network combining spatial–spectral features is often better in HSI classification [9], [42]–[44]. The deep network with pure 1-D and 2-D operators can obtain abundant spectral and spatial information. Therefore, Li *et al.* [40] proposed a 3-DCNN classification method for accurate HSI classification. Similar HSI classification methods based on 3-D deep learning include 3-D DL [45]. According to the experimental results, 3-D convolution is suitable for HSI classification. With the improvement of deep learning methods, the classification accuracy of HSI has been gradually improved, the best classification accuracy is up to 99% with sufficient training samples [46]. However, in practice, it is difficult to obtain a large number of annotations for HSI datasets, which requires a lot of time and labor. Collecting large numbers of training samples may even be impossible in some cases.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:52:12 UTC from IEEE Xplore. Restrictions apply.

Deep learning does not perform well in the absence of data. Moreover, the deep learning model is computation-intensive and the model scale is huge, which makes it difficult to deploy on small mobile devices that need fast calculation. Therefore, traditional methods of deep structure can effectively solve the problem of large model scale and large amount of annotated samples.

C. Few-Shot Learning Methods

In this article, our main aim is to solve the classification problem of hyperspectral images in the case of scarcity of labeled samples. Deep learning use transfer learning to solve this problem, including instance-transfer [47], feature-representation-transfer [48], model-transfer [49], [50] and relational-knowledge-transfer [51]–[53]. The performance of transfer learning is closely dependent on the ability of the pretraining model. At present, transfer learning is often combined with other learning methods, such as reinforcement Transfer Learning [54], adversarial transfer learning [55], lifelong transfer learning [56] and so on. Transfer learning is one of the solutions to the problem of spectral variation of hyperspectral data over time. PCTL-SAS [57] provides a elegant model for efficient transfer learning of HSI data through domain-invariant shared internal representation of shared abundance space. However, it is difficult to obtain sufficient training to obtain a good pretraining model with small samples, and the training of the pretraining model has consumed a huge amount of time and computing resources. Compared with transfer learning, few-shot learning requires only a small amount of data with only a few or a few dozen samples per category to train the model, eliminating the high costs associated with data collection and labeling. Dirichlet-net based on multitask transfer learning [58] is applied to few-shot learning to achieve fast and accurate domain knowledge transfer and greatly improve the algorithm performance.

Mainstream few-shot learning has three aspects, including model-based [59], metric-based [60]–[62] and optimization-based [63]. The model-based method aims to directly establish the mapping function of input and predicted value through the design of model structure. Metric-based method is used to measure the distance between the samples in the support set and the samples in the query set. Optimization-based method performs the task of small sample classification by adjusting optimization methods such as gradient descent. Meta-learning is a general framework of few-shot learning to solve small sample problems. MAML [64] is a successful example of this approach. This is a few-shot learning method based on optimization, which calculates gradients and update parameters in two different datasets, respectively. The meta-learning method enables the model to adjust parameters in the iterative episodes so that the model learns to adapt to small samples, which can reduce overfitting.

D. Few-Shot Hyperspectral Classifiaction

Inspired by the success of few-shot learning [60], [62], [63], some few-shot learning methods are proposed for hyperspectral image classification. It is a main research idea to improve the above few-shot learning method based on deep learning and extend it to hyperspectral classification. Relation Network [65] have been introduced into hyperspectral classification. DFSL method builds a spatial–spectral network based on prototypical network [60]. Similar efforts include [66], which are to learn the prototype center of data in metric space and predict unknown categories in clustering. On this basis, the feature extraction network was designed to improve the classification performance. Spectral–spatial attention network (SSAN) [67] is a dense connected CNN network with attention mechanism was designed to realize the adaptive of different receptive fields. In fact, most of the research on hyperspectral few-shot learning focuses on the design of network structure. 3-DCSN [68] design a 3-D Siamese network structure that combines contrast information with label information to solve the problem of data shortage. Li *et al.* [69] proposed a deep cross-domain few-shot learning (DCFSL) method to deal with the few-shot learning, and they consider the adaptability to HSI domain shift. Wang *et al.* [70] proposed heterogeneous few-shot learning (HFSL) for HSI classification. Effective knowledge transfer between two heterogeneous datasets can significantly improve the accuracy of HSI classification. Spectral spatial residual network (SSRN) [46] uses jump connections to transfer shallow features backward to improve the accuracy of HSI classification. Based on this, Zhong *et al.* [71] proposed a new spatial–spectral transformer network (SSTN), which solved the problem of long-distance feature interaction and improved the classification accuracy of the network.

The feature extraction networks of these methods are obtained from the promotion of deep learning, and the deep learning networks of different structures result in the large scale of the model and high training time cost. Moreover, when there is only a small amount of training data, the model obtained based on deep learning is prone to overfitting. The accuracy of current few-shot learning methods in hyperspectral classification still needs to be improved without overfitting. It is a great challenge for hyperspectral classification to design a lightweight high-precision network with small samples. Aiming at the above requirements, we propose a novel 3-DCRFF network to perform HSI classification where image annotations are scarce.

III. Proposed Framework

Deep learning requires a large amount of labeled data, and insufficient samples will cause serious overfitting. We propose a deep architecture based on the kernel method to solve the problem of few-shot classification. Fig. 1 shows the framework of the proposed 3-DCRFF method in the training phase. 3-DCRFF algorithm is mainly composed of three parts, first, a feature extraction network is designed. Then the meta-amortization variational inference model is connected. Finally, the synthetic gradient is used to solve the network parameters. We will describe each of these components in the next sections. We introduce the RFF kernel approximation strategy, construct the 3-DCRFF basic unit combined with 3-D group convolution, and construct a 20-layer network with skip connections to extract the spatial–spectral features of HSI.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:52:12 UTC from IEEE Xplore. Restrictions apply.

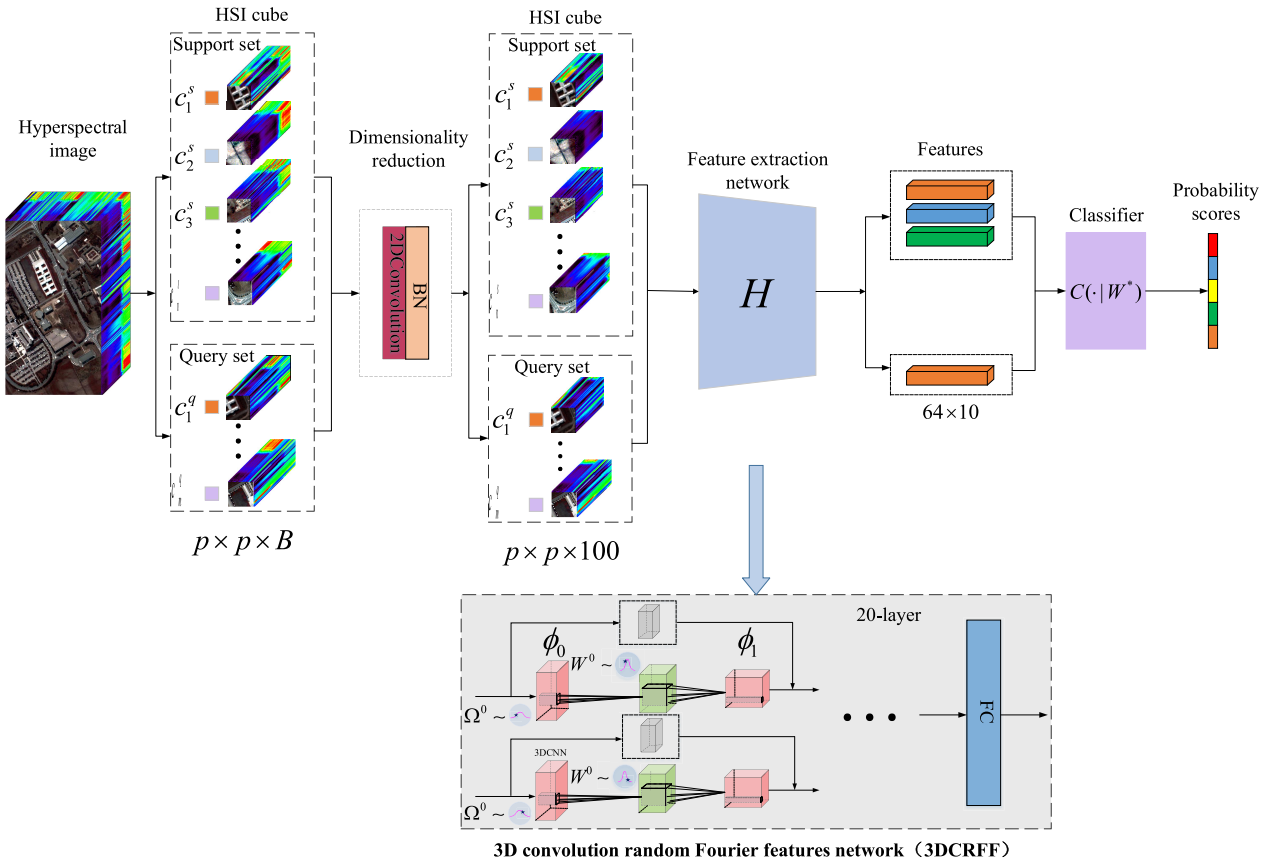


Fig. 1. Overall framework of the 3-DCRFF few-shot learning algorithm. In the figure, $S_1, \ldots, S_k$ represents k samples collected for each class from the support set. A total of C classes were sampled. Likewise, $Q_1, \ldots, Q_m$ represents m samples collected from the query set. The spatial size of the HSI cube is $p \times p \times B$, which B is the number of bands. First, HSI cube is unified into dimension $p \times p \times 100$ after dimension reduction of 2-D convolution of 1*1 kernel size. Second, the HSI cube was fed into the 3-DCRFF network H to extract features and obtain batch $\times$ mc $\times$ 64 features, where $\phi(x)$ represents the explicit mapping function of the RFF approximate kernel function and Ω is the spectral frequency related with the kernel hyperparameters. The details of the 3-DCRFF network are covered in the section III. Finally, mc features sampled by Monte Carlo are averaged before being fed into classifier $C(\cdot|W^*)$.

Network details are shown in the framework of the dotted box in Fig. 1.

A. 3-DCRFF Network

To learn an appropriate embedded feature for HSI few-shot learning task, a meta-learning strategy is adopted. Considering hyperspectral application scenarios, in the first phase, we have the data and resources to train our feature extractor on a large labeled dataset. In the second phase, we want to add additional categories to our dataset with minimal computational and data costs, which meet the needs of practical applications. Inspired by the work [72], we train the algorithm in two phases, pretaining and meta-learning. Meta-learning method can learn feature expressions with good generalization, which is beneficial to the subsequent transfer learning of few-shot categories, which meet the needs of practical applications.

We use episode training strategies to train a $\mathcal{C}$-way $\mathcal{K}$-shot task in each iteration. During each iteration, we randomly sampled $\mathcal{C}$ classes, and each class contained $\mathcal{K}$ label samples as support set input. We define the support set as $S = \{(x_i, y_i)\}_{i=1}^{C\times K}$. Meanwhile, we sampled $\mathcal{N}$ unlabeled samples per class from the remainder of the same $\mathcal{C}$ classes to generate a query set, which as $Q = \{(x_i, y_i)\}_{i=1}^{C\times N}$.

We divide the datasets in a train and test splits by randomly sampling over the whole image. The support set and query set are cubes with a patch size $9 \times 9 \times B$ randomly sampled from the original HSI, where B is the spectral dimension. We select $p \times p \times B$ pixels around each pixel to obtain N_{train} random patches, N_{train} is the number of training set. In our algorithm, it works best when p is 9. For pixels distributed at the edges of the image, we obtain cubes by filling in the neighboring blank pixels. All N_{train} random areas are HSI cubes used for training, and C-way K-shot samples randomly sampled from these cubes can be used as the support set. The generation process of HSI cubes are shown in Fig. 2.

Different datasets have different spectral dimensions. A large number of spectral dimensions are redundant, which greatly increases the amount of calculation. We use a 2-D convolution network to extract effective spectra bands of HSI cubes, as shown in Fig. 1. The dimensions of hyperspectral cubes are unified as $9 \times 9 \times 100$.

Through experiments, the network architecture is finally determined as shown in Table I. The network consists of 20 layers. According to the findings in work [38], small receptive fields of 3×3 convolution kernels with deeper architectures generally yield better results. Therefore, we determine that the size of all convolution kernels of the network is

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:52:12 UTC from IEEE Xplore. Restrictions apply.

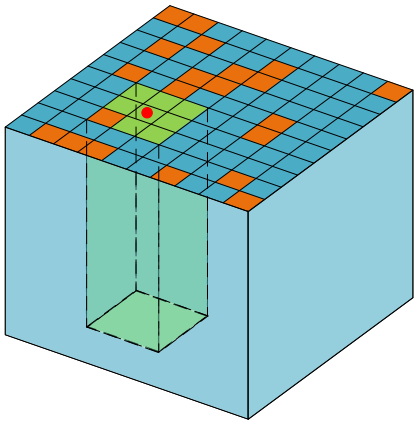


Fig. 2. HSI cube sampled process. Blue is the pixel divided into the training set, orange is the pixel of the test set, and green is the cube block we obtained in the neighborhood with the red dot as the center pixel. There are N_{train} red dots.

$3 \times 3 \times 3$. We specify the number of layers of the network including the RFF layer, the convolution layer, and the fully connected layer except for the bypass concatenation operation(Concat_x). In the table, RFF_x refers to the RFF mapping module, which contains a combination of two operations. First, 3-D group convolution is used to filter the input image vector, and then the features are mapped to the specified feature space by trigonometric function, which is the foundation for the next layer operation. 3-DGConv_x is a pure 3-D group convolution operation. After the mapping of the previous layer, the features are linearly separable. Concat_x is the skip connection of the network. The network architecture proposed by 3-DCRFF algorithm integrates the features between layers by concating the channel dimension, and global average pooling is used on the branch to ensure the consistency of the size of the connection of the feature maps of the upper and lower layers. $k \times k \times k$ is the kernel size of convolution and s is stride. #out represents the number of output channels. The outputs of the final fully connected layer are final feature embedding.

Given the input, we can map the input to a trigonometric function feature space by the RFF functions. In this article, we use deep 3-D convolutional RFF to approximate the kernel function. Skip connections that fall off the network using global average pooling.

We take the d-dimensional RBF function as the base kernel

$$k(x, x^\top) = \sigma^2 \exp\left(-\frac{1}{2}(x - x^\top)^\top \Gamma^{-1}(x - x^\top)\right) \tag{1}$$

with $x = [x_1^s, \ldots, x_i^s]$ and $\theta = \{\sigma^2, \text{diag}(\gamma_1^2, \ldots, \gamma_d^2))\}$. Based on Bochner's theorem [73] and Monte Carlo sampling, the feature representation of the $2M$-dimensional space can be a formula as

$$\phi(x) = \sqrt{\frac{\sigma^2}{M}}\left[\cos\left(\omega_1^\top x\right), \sin\left(\omega_1^\top x\right), \ldots, \cos\left(\omega_M^\top x\right), \sin\left(\omega_M^\top x\right)\right]^\top. \tag{2}$$

In the new feature space, we can use linear learning methods to approximate nonlinear learning algorithms. The linear function is defined as follows:

$$F(x) = w^\top \phi(x), \quad \text{with } w \sim \mathcal{N}(0, I). \tag{3}$$

Based on work [74], [75], it is theoretically possible to construct a multilayer deep kernel network by cascading the above functions. For a L-layer deep kernel learning network, the features extraction function is expressed as

$$F^{(l+1)} = {W^{(l)}}^\top \phi(F^{(l)}; \Omega^{(l)}), \quad \text{for } l = 0, \ldots, L-1 \tag{4}$$

where $\Omega^{(l)} = \{\omega_1^{(l)}, \ldots, \omega_M^{(l)}\}$, and $W^{(l)} \sim \mathcal{N}(0, I)$, $\Omega^{(l)} \sim \mathcal{N}(0, (\Gamma^{-1})^{(l)})$. In order to fully learn the spatial and spectral features of HSI 3-D data, we extend the 2-D matrix operation to 3-D convolution to construct a deep 3-DCRFF network and realize the construction of a deep kernel learning feature extraction network. The feature extraction function is

$$H(x; \Theta, \Omega, W) = h^{(L-1)}(\cdots(h^{(1)}(h^{(0)}(x)))) \tag{5}$$

where $\Theta = \{\theta^{(l)}\}_{l=1}^L$ is the parameter set of kernel functions $\theta^{(l)} = \{(\sigma^2)^{(l)}, (\Gamma^{-1})^{(l)}\}$, $\Omega = \{\omega^{(l)}\}_{l=1}^L$ is the spectral frequency parameter, and $W = \{w^{(l)}\}_{l=1}^L$ is the weight parameter of the Bayesian linear model (RFF_x).The mapping function is

$$\phi(x) = \sqrt{\frac{\sigma^2}{M}}\left[\cos\left(\omega_1^\top \otimes x\right), \sin\left(\omega_1^\top \otimes x\right), \ldots, \cos\left(\omega_M^\top \otimes x\right), \sin\left(\omega_M^\top \otimes x\right)\right]^\top \tag{6}$$

where the operator $\otimes$ represents 3-D group convolution. We use grouping to implement Monte Carlo sampling in

TABLE I
NETWORK OF 3-DCRFF FEATURE EXTRACTOR

Input Size	Operator	$k \times k \times k/s$	#out
$9 \times 9 \times 100$	RFF_1	$3 \times 3 \times 3/s1$	2
$9 \times 9 \times 100$	3DGConv_1	$3 \times 3 \times 3/s1$	2
$9 \times 9 \times 100$	RFF_2	$3 \times 3 \times 3/s1$	2
$9 \times 9 \times 100$	3DGConv_2	$3 \times 3 \times 3/s1$	2
$9 \times 9 \times 100$	Concat_1	-	4
$9 \times 9 \times 100$	RFF_3	$3 \times 3 \times 3/s1$	2
$9 \times 9 \times 100$	3DGConv_3	$3 \times 3 \times 3/s1$	2
$9 \times 9 \times 100$	Concat_2	-	4
$9 \times 9 \times 100$	RFF_4	$3 \times 3 \times 3/s2$	4
$5 \times 5 \times 50$	3DGConv_4	$3 \times 3 \times 3/s1$	4
$5 \times 5 \times 50$	Concat_3	-	6
$5 \times 5 \times 50$	RFF_5	$3 \times 3 \times 3/s1$	4
$5 \times 5 \times 50$	3DGConv_5	$3 \times 3 \times 3/s1$	4
$5 \times 5 \times 50$	Concat_4	-	8
$5 \times 5 \times 50$	RFF_6	$3 \times 3 \times 3/s2$	8
$3 \times 3 \times 25$	3DGConv_6	$3 \times 3 \times 3/s1$	8
$3 \times 3 \times 25$	Concat_5	-	12
$3 \times 3 \times 25$	RFF_7	$3 \times 3 \times 3/s1$	8
$3 \times 3 \times 25$	3DGConv_7	$3 \times 3 \times 3/s1$	8
$3 \times 3 \times 25$	Concat_6	-	16
$3 \times 3 \times 25$	RFF_8	$3 \times 3 \times 3/s2$	16
$2 \times 2 \times 13$	3DGConv_8	$3 \times 3 \times 3/s1$	16
$2 \times 2 \times 13$	Concat_7	-	24
$2 \times 2 \times 13$	RFF_9	$3 \times 3 \times 3/s1$	16
$2 \times 2 \times 13$	3DGConv_9	$3 \times 3 \times 3/s1$	16
$2 \times 2 \times 13$	Concat_8	-	32
$2 \times 2 \times 13$	RFF_10	$3 \times 3 \times 3/s1$	16
$2 \times 2 \times 64$	FC	-	64

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:52:12 UTC from IEEE Xplore. Restrictions apply.

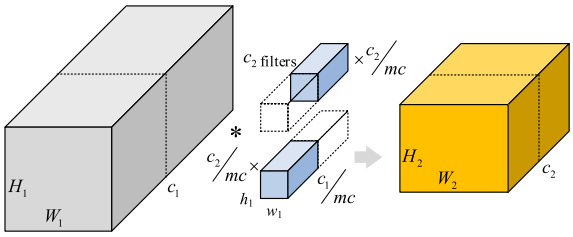


Fig. 3. 3-D group convolution operation.

the algorithm. The convolution operation is simplified in Fig. 1. 3-D group convolution in our algorithm is shown in Fig. 3. We divide the input HSI cube into mc groups, corresponding mc group 3-D convolution kernel. mc is the number of Monte Carlo sampling. The introduction of 3-D group convolution enables the network to effectively fuse RFF and realize fast information extraction of HSI cubes.

B. Classifier Based on Cosine Similarity

Motivated by the study [76], we use classifier $C(\cdot|W^*)$, and the score of the network classification is obtained by the cosine similarity operator

$$p_k = \tau \cdot \cos\left(f, w_k^*\right) = \tau \cdot \bar{f}^\top \bar{w}_k^* \tag{7}$$

where $\bar{f} = (f/\|f\|)$ and $\bar{w}_k^* = (w_k^*/\|w_k^*\|)$. τ is initialized to 10.

We infer the classification weight vector w_k^* by averaging the feature vectors of the input samples. After obtain the input feature vectors $F = \{f_a\}_{a=1}^{n_s}$, we obtain average classification weights based on category clustering

$$w_{\text{avg}}^* = \frac{1}{n_s}\sum_{a=1}^{n_s} \bar{f}_a \tag{8}$$

where n_s is the number of input samples. The weight vector is regarded as the initialization of the synthetic gradient below, through which the weights are generated for the query set, the network can be better generalized to the new class.

In the synthetic gradient, we train w_{avg}^* for m times to get w_m

$$w_m^* = \frac{1}{m}(\phi_{\text{avg}} \odot w_{\text{avg}}^* + \phi_k \odot w_k^*, \quad \text{with } k = 1, 2, \ldots, m) \tag{9}$$

where ϕ_{avg} and ϕ_i is learned scale vector with the same size as the weight vector. There are m w_i, and it can produce m models along the training period of each episode. The final prediction feature vector is the average of the prediction results of all models which called ensemble model. Obtaining the weight vector and input feature representation can return K probability classification fractions $p = C(\cdot|W^*)$, where W^* is a set of weight vectors and $C(\cdot)$ is the classifier. The log-likelihood model is generated by minimizing the cross entropy loss function. Inspired by work [77], 3-DCRFF algorithm uses meta-amortization variational inference method to solve parameters.

C. Meta-Amortized Variational Bayes and Learning

In practice, for the dataset $D = \{S, Q\}$, we implement a meta-inference model for few-shot learning. During training, we optimize Θ to jointly learn the posterior over Ω and W. Based on the mean-field theory, we introduce a variational density $q(\Psi|S, \Theta)$ to approximate the posterior $p(\Psi|x, y, S, \Theta)$ of the parameters to be learned. Where y is the label, $\Psi = \{\Omega, W\}$ is the variational parameters. We maximize the logarithmic marginal likelihood of the data as follows:

$$\max_{\Psi} E_{q_\Psi} \ln \frac{p(\Psi|S)}{q(\Psi|S, \Theta)}. \tag{10}$$

The above formula is difficult to solve, so we use $r(x_i)$ as a tractable amortization inference model to deduce the evidential lower bound of the above formula as follows:

$$\log p(Y|X, S, \Theta) \geq E_{q(\Psi|S)}[\log p(Y|X, S, \Psi, \Theta)] - D_{\text{KL}}(q(\Psi|S)|p(\Psi|S, \Theta)]. \tag{11}$$

We derive a doubly stochastic approximate of the expectation by using the mini-batch and reparameterized Monte Carlo sampling technique [78]. The decomposed expectation is denoted as

$$\frac{N}{m}\sum_{m}\frac{1}{N_{\text{MC}}}\sum_{r=1}^{\text{MC}}\log p(y_k|x_k, S, \hat{\Psi}_r, \Theta) \tag{12}$$

where $\hat{\mathbf{\Psi}}_r = \{\hat{\mathbf{\Omega}}_r, \hat{\mathbf{W}}_r\}$ are obtained by sampling from a [0, 1] normal distribution. where

$$\hat{\boldsymbol{\omega}}_{ij}^{(l)} = \mu_{ij}^{(l)} + \beta_{ij}^{(l)}\varepsilon_{ij}^{(l)}, \quad \text{with } \varepsilon_{ij}^{(l)} \sim \mathcal{N}(0, 1) \tag{13}$$

$$\hat{\mathbf{w}}_{ij}^{(l)} = m_{ij}^{(l)} + s_{ij}^{(l)}\tau_{ij}^{(l)}, \quad \text{with } \tau_{ij}^{(l)} \sim \mathcal{N}(0, 1). \tag{14}$$

Maximizing the right ELBO is equivalent to minimizing the discrepancy between the entropy of variational distribution $q(\Psi|S)$ and the prior $p(\Psi|S, \Theta)$. The focus of network optimization is to minimize KL divergence of T tasks

$$\min_{\Psi}\min_{\Theta}\frac{1}{T}\sum_{t=1}^{T} D_{\text{KL}}(q(\Psi|S)|p(\Psi|S, \Theta)). \tag{15}$$

We design a 3-layer multilayer perceptron (MLP) $r(x_i)$ to model the variational parameters of each distribution. The framework of the three-layer perceptron is shown in Fig. 4. The proposed algorithm learns potentially transferable representations that can be extended across multiple related distributions. In particular, given a set of distributions on the image, the proposed algorithm finds learning representations to migrate to different data transformations.

In our algorithm, we optimize the parameters by synthetic gradients to speed up the training. We define φ_t^k is a function of Ψ, Θ and the parameter W^* of $f(x_i)$ and φ_t^k is the average weight vector. The parameter update step can be simplified and formulated as

$$\varphi_t^{k+1} = \varphi_t^k - \lambda\nabla_{\Psi,\Theta,W^*} D_{\text{KL}}(q_{f(x_i)}(\Psi|S)|p(\Psi|S, \Theta)). \tag{16}$$

We dynamically optimize the network parameters until φ_t^k approximate the optimal parameters. In our algorithm, we set $k = 3$.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:52:12 UTC from IEEE Xplore. Restrictions apply.

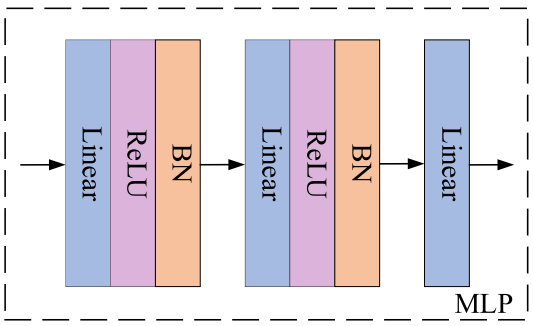


Fig. 4. 3-layer MLP.

TABLE II
HYPERSPECTRAL IMAGE DATASETS FOR EVALUATION

	UP	PC	IP	Salinas
Sensors	ROSIS	ROSIS	NASA AVIRIS	NASA AVIRIS
Spatial dimensions	610×340	1096×715	145×145	512×217
Spectral bands	103	102	200	204
Wavelength range(nm)	430∼860	430∼860	400∼2500	360∼2500
Spatial Resolution(m)	1.3	1.3	20	3.7
Labels	42776	7456	10249	54129
Classes	9	9	16	16
Mode	Aerial	Aerial	Aerial	Aerial

In summary, we propose a new feature extraction network in this article, which is composed of 3-DCRFF. Combined with the adaptability of kernel learning to small samples and the powerful feature extraction capability of deep learning deep structure, the proposed network can effectively extract spatial and spectral information of HSI cubes. Meta-amortized variational inference is used to transfer information between different distributions, and the synthetic gradient is used to optimize network parameters quickly.

IV. DATASETS AND SETTINGS

A. Datasets

To verify the performance of our proposed algorithm, we conduct experiments on four public datasets, including the University of Pavia (UP), the Pavia Center (PC), the Indian Pines (IP), and the Salinas Valley (Salinas). In the experiment, we randomly divided the dataset into a training set and test set according to 50%. The training set is used to pretrain the feature extraction network, and the test set is further divided for meta-learning to train classifier. We summarize the descriptions of four datasets in Table II.

1) UP: The UP dataset was acquired in 2001 using the Reflective Optics Spectrographic Image System (ROSIS) in the cities of Pavia, northern Italy. The spatial dimension of the image is 610 × 340 and contains 103 bands with wavelengths ranging from 430 nm to 860 nm. The band region of water absorption was discarded for analysis. The number of labeled pixels was 42 776, including nine valid categories. Fig. 5 shows the false-color map of the UP dataset and the corresponding ground truth map. The undefined class is the abandoned band area marked black. The specific land cover category and the corresponding number of labeled pixels are shown in Table III. The meadows class occupies the largest area.

2) PC: Like the UP dataset, the PC dataset was acquired by ROSIS sensors. It contains a total of 7456 labeled samples.

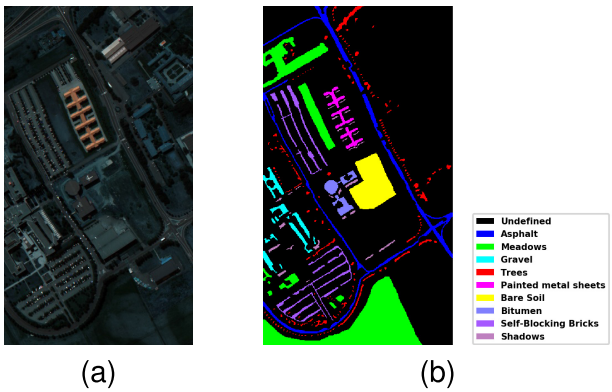


Fig. 5. UP dataset. (a) False-color image (55, 41, 12). (b) Groundtruth map.

TABLE III
LAND-COVER CLASSES AND SAMPLES FOR THE UP DATASET

Label	Class	Samples
1	Asphalt	6631
2	Meadows	18649
3	Gravel	2099
4	Trees	3064
5	Painted metal sheets	1345
6	Bare Soil	5029
7	Bitumen	1330
8	Self-Blocking Bricks	3682
9	Shadows	947

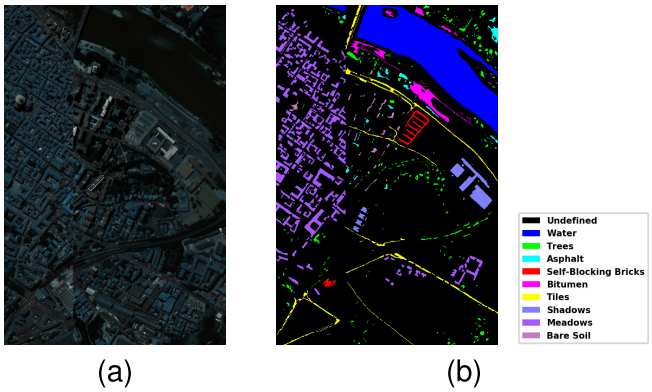


Fig. 6. PC dataset. (a) False-color image (55, 41, 12). (b) Groundtruth map.

The spatial dimension of the image is 1096 × 715, and the number of spectral bands after removing the water absorption region is 102. We visualized the pseudo-color image and the corresponding ground truth map, as shown in Fig. 6. The ground truth contains nine different typical urban site classes. Detailed categories and corresponding sample numbers are shown in Table IV.

3) IP: The IP dataset was acquired by NASA's Airborne Visible/Infrared Imaging Spectrometer (AVIRIS) sensors in 1992. The image covers the IP Test site in North West Indiana. False-color image and ground truth map in our experiments are shown in Fig. 7. The image contains 16 classes. The spatial size of the image is 145 × 145 pixels, its spatial resolution is 20 m, including 200 spectral bands excluding water absorption, and the wavelength range is 400 nm to 2500 nm. The total number of labeled samples in the IP is

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:52:12 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
LAND-COVER CLASSES AND SAMPLES FOR THE PC DATASET

Label	Class	Samples
1	Water	824
2	Trees	820
3	Asphalt	816
4	Self-Blocking Bricks	808
5	Bitumen	808
6	Tiles	1260
7	Shadows	476
8	Meadows	824
9	Bare Soil	820

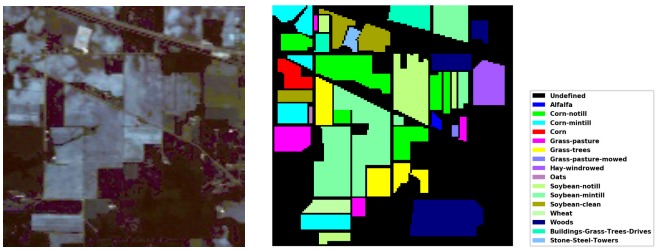


Fig. 7. IP dataset. (a) False-color image (29, 19, 9). (b) Groundtruth map.

TABLE V
LAND-COVER CLASSES AND SAMPLES FOR THE IP DATASET

Label	Class	Samples
1	Alfalfa	46
2	Corn-notill	1428
3	Corn-mintill	830
4	Corn	237
5	Grass-pasture	483
6	Grass-trees	730
7	Grass-pasture-mowed	28
8	Hay-windrowed	478
9	Oats	20
10	Soybean-notill	972
11	Soybean-mintill	2455
12	Soybean-clean	593
13	Wheat	205
14	Woods	1265
15	Buildings-Grass-Trees-Drives	386
16	Stone-Steel-Towers	93

10 249, and the number of labeled samples corresponding to each category is shown in Table V.

4) Salinas: The Salinas dataset was collected in 1998. The data were captured by AVIRIS sensors flying over Salinas Valley, CA, USA. The image spatial dimension is 512 × 217 with 204 spectral bands excluding water absorption bands in the 360 nm 2500 nm wavelength range. A total of 54 129 labeled samples covered 16 classes in the SA dataset. The false-color image and the corresponding ground truth map are shown in Fig. 8. Table VI shows the corresponding number of labels for each category of samples.

B. Experiment Settings

Our experiments were run using PyTorch on a server with NVIDIA GTX 1080Ti GPU and 24G RAM. The network consists of two parts for feature extraction and classification,

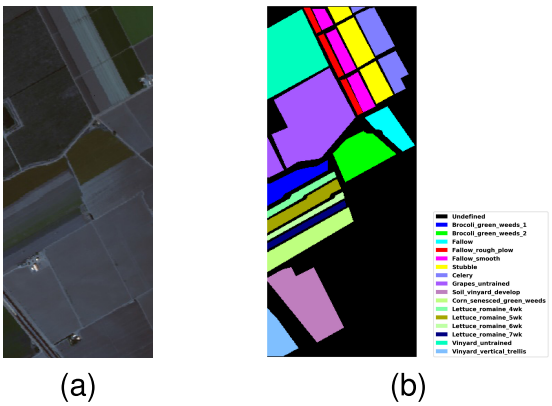


Fig. 8. Salinas dataset. (a) False-color image (29, 19, 9). (b) Groundtruth map.

TABLE VI
LAND-COVER CLASSES AND SAMPLES FOR THE SALINAS DATASET

Label	Class	Samples
1	Brocoli_green_weeds_1	2009
2	Brocoli_green_weeds_2	3726
3	Fallow	1976
4	Fallow_rough_plow	1394
5	Fallow_smooth	2678
6	Stubble	3959
7	Celery	3579
8	Grapes_untrained	11271
9	Soil_vinyard_develop	6203
10	Corn_senesced_green_weeds	3278
11	Lettuce_romaine_4wk	1068
12	Lettuce_romaine_5wk	1927
13	Lettuce_romaine_6wk	916
14	Lettuce_romaine_7wk	1070
15	Vinyard_untrained	7268
16	Vinyard_vertical_trellis	1807

respectively. To obtain better classification results, we first pretrain the feature extraction network, and then train the whole network by freezing parameters. In the pretraining process, we set the number of training epochs of the network as 120 to make the network reach a certain convergence, and the learning rate is 0.001. The learning rate is attenuated according to the simulated annealing strategy. The batch size is 64. We use SGD optimizer to optimize the parameters. The feature extraction network is composed of the proposed 3-DCRFF, the base kernel function is the arccos function, and the Monte Carlo sampling times are set to 5. For hyperspectral images, we randomly cut the image into 9 × 9 spatial patch size as the network input cube-like [69]. In a few-shot classification, we run 50 000 iterations to implement network learning. Each iteration consists of a C-way K-shot episode. C represents the number of categories used for training, and K represents the number of samples extracted from per class. On different datasets, the results of the C-way 5-shot were compared as benchmarks. For UP and PC dataset, C is set to 9, otherwise to 16. In the test, the higher the number of query sets, the better the verification effect of small sample classification. Based on experience [38], [65], we set the number of query sets to 19. For classification, we set the number of optimization dynamics is 3.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:52:12 UTC from IEEE Xplore. Restrictions apply.

C. Metrics

In our experiment, we used three commonly used metrics as evaluation criteria for network performance, including overall accuracy (OA), average accuracy (AA), and Kappa coefficient (kappa).

1) *OA:* OA is the ratio of the number of correct predictions made by the model to the total number of samples on all test sets. The calculation formula is as follows:

$$\mathrm{OA} = \frac{N_c}{N_{\text{total}}} \tag{17}$$

where N_c represents the number of samples correctly classified by the model. N_{total} represents the total number of all samples used for evaluation.

2) *AA:* The AA is the ratio of classification accuracy per class to the total number of classes in the dataset. It represents the AA per class. The detailed formula is defined as follows:

$$\mathrm{AA} = \frac{\sum_{i=1}^{n} acc_i}{n} \tag{18}$$

where n is the total number of categories in the classification. we define acc_i as the corresponding classification accuracy of class i.

3) *Kappa:* Kappa coefficient is a more accurate measure of classification accuracy and can better reflect the consistency between the real value and the predicted value. The result is formulated as

$$\mathrm{kappa} = \frac{p_o - p_e}{1 - p_e} \tag{19}$$

where p_o is observation agreement, which is the overall classification accuracy of the model, and p_e is the expected agreement between actual ground truth and predicted values of the model for all categories.

V. Experiment Results and Analysis

A. Ablation Study

In Table VII, we perform an ablation experiment to prove that our algorithm is effective in improving classification performance. We mainly consider the influence of two structures on classification performance. **A** is the RFF network with convolution operation, and **B** is the ensemble model with synthetic gradient. We use "×" for removing component, otherwise using the component. When the convolution (**A**) in the network is removed, the basic neurons of our network degenerate into traditional RFF (RFF) [79], and the network is composed of DNNs. For RFF networks, the network will diverge when stacked to layer 20. Therefore, we choose an effective RFF network structure [80] for comparison. When the synthetic gradient (**B**) is removed, our parameter optimization algorithm degenerates into standard amortization variational inference. To prove that our proposed network framework with 3-DCRFF using synthetic gradient is valid, we verify the classification performance of different structures on the UP dataset.

Considering both experiment 1 and experiment 2, only the feature extraction network was changed to the RFF with convolution, and the classification accuracy of the model is greatly improved by 4.46%. The experimental results proved the effectiveness of the convolution RFF. According to experiments 1 and 3, amortization variational inference with synthetic gradient (**B**) can make the network learn a better classification model. The overall classification accuracy of the network in experiment 3 was improved by 3.61% compared with that in experiment 1, which reflects the rationality and effectiveness of the variational inference with synthetic gradient introduced in our method. In experiments 4 (3-DCRFF), we use both the feature extraction network with convolution RFF and the variational inference optimization algorithm with synthetic gradient, and the classification accuracy exceeds that of the model using only one of the components, which fully demonstrates the superiority of 3-DCRFF proposed by us.

Based on the above analysis, our algorithm is reasonable and effective.

TABLE VII

Classification Performance (in %) on UP Dataset With Different Structures

Experiment	**A**	**B**	OA	AA	Kappa
1	×	×	88.41±1.30	87.16±0.92	84.68±1.65
2	✓	×	92.87±1.16	89.72±1.42	90.51±1.55
3	×	✓	92.02±1.84	88.74±2.03	89.33±2.45
4	✓	✓	**93.35±1.34**	**90.42±1.29**	**91.20±1.73**

TABLE VIII

Performance (in %) on UP Dataset With Different Network Width

Width factor	1	2	4	8
1	83.04±8.42	88.40±5.48	80.57±7.36	93.71±6.74
2	99.35±0.74	96.59±0.35	90.41±1.95	91.71±4.45
3	79.56±6.01	78.03±4.71	78.37±9.22	86.30±2.53
4	99.35±2.14	87.06±4.21	79.57±9.30	95.35±2.62
5	99.48±4.16	99.78±0.35	65.87±9.24	99.36±0.49
6	96.14±2.92	92.13±7.35	94.09±6.11	86.77±1.71
7	78.34±9.18	66.79±8.49	66.01±9.52	88.26±2.17
8	83.25±6.49	66.68±9.07	19.99±9.33	82.00±9.70
9	95.33±3.76	88.92±7.80	98.02±1.46	98.56±2.04
OA	**93.35±1.34**	89.64±1.81	80.53±1.71	90.88±1.25
AA	90.42±1.29	84.93±2.95	74.77±8.13	**91.34±2.48**
kappa	**91.20±1.73**	86.29±2.38	74.53±2.54	88.07±1.71

B. Impact of Different Parameters

In this section, we analyze the influence of different parameter settings on network performance, mainly considering the parameters including the depth and width of the feature extraction network and the size of the input sample cube of the network. In the following experiments, we verify and compare the model classification performance on different parameters on the UP dataset.

1) Network Width: The width of the network refers to the number of convolution kernels at each layer of the feature extraction network. In our network design, the width of the first layer of the network is 2 and then doubled when the size of the feature map is reduced. We test different networks by setting the width factor. The width factor ranges from [1, 2, 4, 8], corresponding to the width of the first layer of the network is 2, 4, 8, and 16. The experimental results are the classification mean and standard deviation of ten runs, as shown in Table VIII.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:52:12 UTC from IEEE Xplore. Restrictions apply.

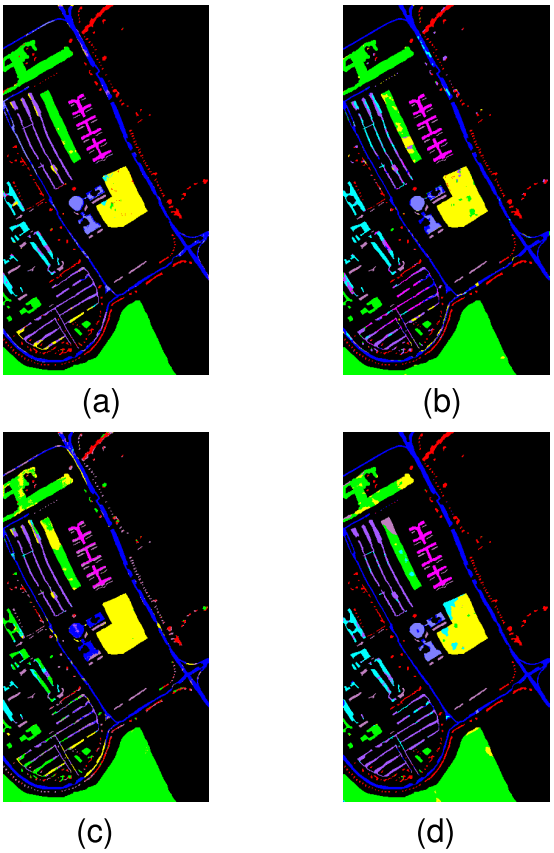


Fig. 9. Map of network classification results with different width factor. (a) Factor = 1. (b) Factor = 2. (c) Factor = 4. (d) Factor = 8.

As can be seen from the table, when the width factor is 1, the overall classification accuracy is as high as 93.35% and the average classification accuracy is 90.42%, indicating that the wider the network width is not the better. When the network width multiplier is 2, the network performance decreases by about 4%, and when the network width multiplier is 4, the classification performance decreases sharply by as much as 13%. Although the performance improved when the width multiplier was 8, it still could not achieve the network effect we designed. Moreover, increased width introduces more computation, resulting in memory resource consumption. The land cover of network classification results on different width designs is shown in Fig. 9. It is obvious from the figure that the classification effect is best when the width factor is 1. The network we design is a tradeoff between accuracy and efficiency in terms of width design.

2) Network Depth: We conducted experiments under different network depth designs to investigate the effect of depth on model classification performance. We train and test on 5 networks with depths 6, 8, 12, 16, and 20. The average results of ten independent experiments are recorded as shown in Table IX. It is obvious from the results in the table that increasing the depth of the network can achieve better classification performance. We regard a RFF layer and a common convolution layer in the network as a group, and the depth of the network is always a multiple of 2. Considering the computation cost of the network, we set the maximum of the network to be no more than 20 layers. We chose the 20-layer network with the best-classified performance as the default setting for our all experiments.

TABLE IX

PERFORMANCE (IN %) ON UP DATASET WITH DIFFERENT NETWORK DEPTH

Network depth	6	8	12	16	20
1	61.25±2.91	64.42±5.04	76.24±4.38	81.90±3.28	83.04±8.42
2	93.68±3.14	93.00±4.77	99.16±0.70	94.43±4.09	99.35±0.74
3	56.97±9.60	59.00±6.30	58.87±6.80	76.63±8.39	79.56±6.01
4	86.05±8.93	70.39±8.98	81.03±9.66	86.15±9.84	99.35±2.14
5	83.87±8.69	96.19±2.31	78.96±5.66	96.12±5.02	99.48±4.16
6	71.65±5.75	76.32±5.78	80.29±7.47	78.94±3.27	96.14±2.92
7	72.06±3.47	71.23±6.95	66.76±3.60	83.98±3.39	78.34±6.18
8	40.64±4.52	40.84±2.16	69.80±6.84	69.89±2.97	83.25±6.49
9	50.11±4.90	87.32±1.07	76.20±7.55	75.54±3.99	95.33±3.76
OA	77.21±6.09	81.02±2.56	85.44±3.05	89.68±2.54	**93.35±1.34**
AA	68.47±6.20	73.19±4.65	76.37±6.52	82.62±5.80	**90.42±1.29**
kappa	69.73±8.59	71.02±3.24	80.30±4.38	82.05±3.35	**91.20±1.73**

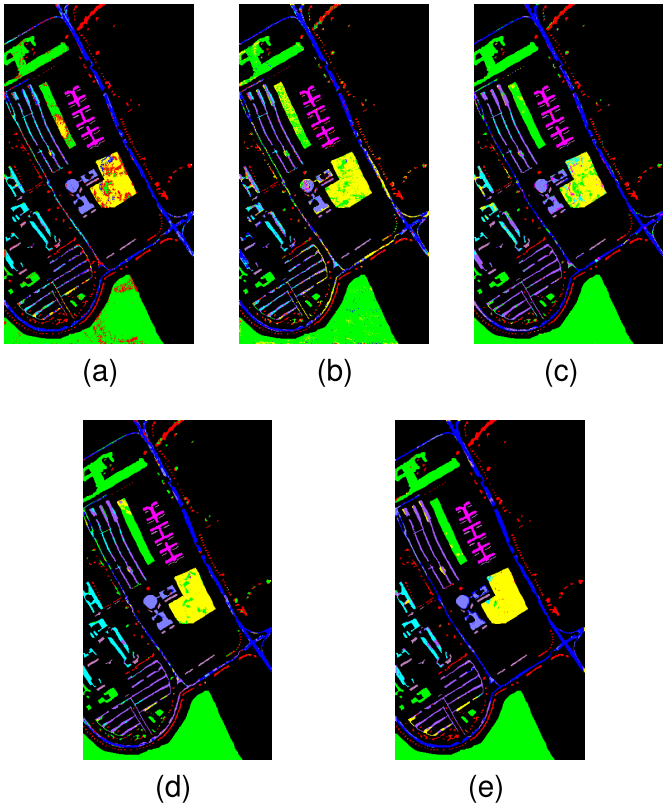


Fig. 10. Map of network classification with different depths. (a) $d = 6$. (b) $d = 8$. (c) $d = 12$. (d) $d = 16$. (e) $d = 20$.

The land cover classification mapping results of the network are shown in Fig. 10. We define d as the depth of the network. The results in the figure directly reflect the overwhelming advantage of the 20-layer network over other network settings. The network performed particularly well in Meadows (green), Trees (red), and Painted metal sheets (pink) categories, with classification accuracy of more than 99%.

3) Patch Size: High-dimensional hyperspectral images are input into the network through random sampling of small 3-D data cubes of size $c \times c \times B$, which c is the spatial size of input samples and B is the number of bands. Set the value c so that all datasets can use the same architecture of the design network. We train and test the classification performance of the network under different input patch sizes, and the value range of patch size is [7, 9, 11, 13, 15].

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:52:12 UTC from IEEE Xplore. Restrictions apply.

TABLE X
PERFORMANCE (IN %) ON UP DATASET WITH DIFFERENT PATCH SIZE

Patch size	7	9	11	13	15
1	76.42±8.48	83.04±8.42	85.72±6.56	89.50±6.08	95.16±2.58
2	95.56±3.91	99.35±0.74	99.41±1.10	88.81±7.20	96.69±4.25
3	26.03±6.54	79.56±6.01	78.19±9.54	52.18±9.93	93.30±2.59
4	69.82±6.66	99.35±2.14	84.07±9.03	65.20±9.94	90.93±1.15
5	84.60±9.06	99.48±4.16	85.52±8.25	92.00±9.94	95.07±1.82
6	83.61±9.47	96.14±2.92	94.87±3.74	93.53±4.77	59.30±9.16
7	52.83±8.38	78.34±6.18	80.69±7.91	20.94±8.36	86.69±5.32
8	23.79±9.51	83.25±6.49	73.18±7.12	69.52±9.01	90.43±3.04
9	65.61±9.24	95.33±3.76	76.03±8.04	72.63±6.96	92.50±3.09
OA	77.43±1.19	**93.35±1.34**	90.83±2.66	81.97±9.43	90.48±0.72
AA	64.25±8.20	**90.42±1.29**	84.19±9.22	71.59±8.23	88.90±2.83
kappa	69.90±1.46	**91.20±1.73**	87.79±3.47	76.92±9.79	87.13±1.21

The experimental results are summarized in Table X. As the patch size of network input increases, the time consumption of the network also increases. Theoretically, the time cost is the lowest when the input size is 7. However, when the network input cube is $7 \times 7 \times B$, the classification performance of the model decreases sharply compared with that of the input sample size is $9 \times 9 \times B$, with a 15.92% difference in classification accuracy. Moreover, increasing the input size of the network does not improve the classification capability of the model. Therefore, considering the compromise between network classification performance and computational efficiency, the input sample is determined to be $9 \times 9 \times B$ data cube.

The classification mapping results with 7, 9, 11, 13, and 15 patch size is shown in Fig. 11. Compared with other networks with input sizes, networks with input sizes of 9 can obtain more accurate results. The classification accuracy of the network is extremely low when the input size is 7 and 13, including class 3 (Gravel), class 7 (Bitumen), and class 8 (Self-Blocking Bricks). When the input size is 11 and 15, the classification accuracy was slightly higher than 90%, but still inferior to the network with the input size of 9. In summary, it is reasonable to determine the spatial input size of the network is 9×9.

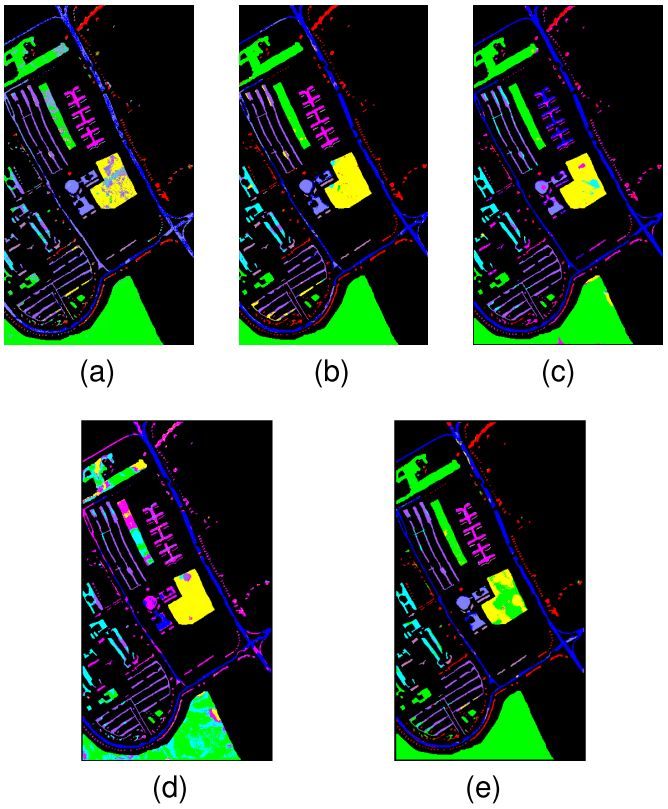


Fig. 11. Map of network classification with different patch size. (a) $p = 7$. (b) $p = 9$. (c) $p = 11$. (d) $p = 13$. (e) $p = 15$.

TABLE XI
CLASSIFICATION RESULTS (IN %) OF THE DEEP 3-DCRFF NETWORK ON DIFFERENT DATASETS

Datasets	UP	PC	IP	Salinas
OA	93.35±1.34	96.94±0.46	70.53±1.89	92.62±1.41
AA	90.42±1.29	84.23±2.47	72.76±1.41	95.23±1.33
kappa	91.20±1.73	94.86±0.65	63.82±1.96	92.00±0.02

C. Performance of the Deep 3-DCRFF Network

The network depth is 20, the width of the first layer is 2, and the size of the input patch is 9×9. To verify the effectiveness of the network, we carry out tests on four widely used benchmarks HSI datasets UP, PC, IP, and Salinas. In the experiment, five samples were randomly sampled from each class for training. UP and PC datasets contained nine classes, while IP and Salinas contained 16 classes. The classification results are shown in Table XI. On UP and PC datasets, the classification accuracy of our algorithm is UP to 93.35% and 96.94%, respectively. Even on Salinas dataset with a higher number of categories, the classification accuracy reached 92.70%. The classification results on these datasets demonstrate the effectiveness of our algorithm. The classification accuracy of the IP dataset is only 70.23%, which is caused by the serious classes imbalance. The number of Oats samples in the IP dataset is only 20, while the class with the maximum number of samples was 2455.

Generally speaking, our algorithm has excellent classification ability on the relatively balanced datasets, and the classification effect of other datasets is also at the advanced level.

D. Comparisons

1) Compared With Other Methods: In this section, to prove the performance of our proposed algorithm, we compare our algorithm with nine hyperspectral image classification algorithms, including SVM [13], KNN [14], SSRN [46], SSTN [71], 3-DCNN [40], 3-D DL [45], 3-DCSN [68], DCFSL [69], and HFSL [70]. We test the classification performance of these algorithms with a limited number of labeled samples. We use publicly available code to conduct a small sample classification comparison experiment on four datasets. Table XII to Table XV is the mean and standard deviation (std.) of classification accuracy obtained from ten runs of all algorithms on dataset UP, PC, IP, and Salinas, respectively. The results in the table are OA, AA, and Kappa coefficients when the number of training samples per class is only 5. The corresponding classification maps are shown in Figs. 13 to 16.

Table XII shows the comparison results on dataset UP. The performance of our algorithm greatly exceeds that of all other algorithms. When the classification accuracy of our algorithm is as high as 93.35%, the classification accuracy

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:52:12 UTC from IEEE Xplore. Restrictions apply.

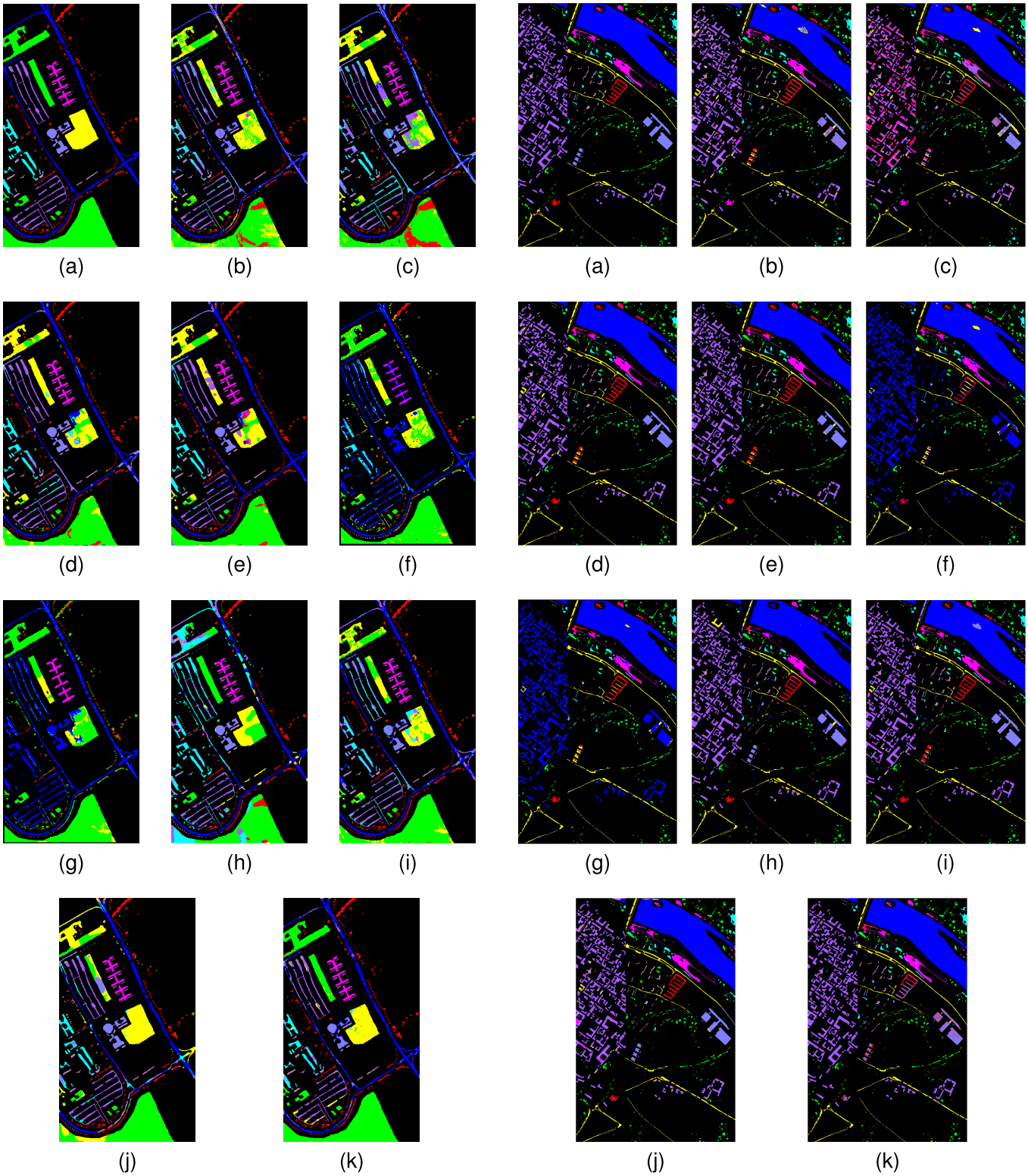


Fig. 12. UP. (a) GroundTruth. (b)–(k) Classification maps for different methods. (b) SVM. (c) KNN. (d) SSRN. (e) SSTN. (f) 3-DCNN. (g) 3-D-DL. (h) 3-DCSN. (i) DCFSL. (j) HFSL. (k) Ours.

Fig. 13. PC. (a) GroundTruth. (b)–(k) Classification maps for different methods. (b) SVM. (c) KNN. (d) SSRN. (e) SSTN. (f) 3-DCNN. (g) 3-D-DL. (h) 3-DCSN. (i) DCFSL. (j) HFSL. (k) Ours.

of other algorithms is less than 85% when the number of samples is small. Our algorithm is 8.58% higher than the most advanced HFSL algorithm. It can also be seen from the Fig. 12 that the classification result of our algorithm is closest to the ground truth. Although the classification results of HFSL appear to be smooth, HFSL does not perform as well as our algorithm on class 1 (Meadows) and class 2 (Asphalt). This shows the effectiveness of our algorithm in small sample HSI classification.

Similar to the UP dataset, the PC dataset contains nine typical city categories, but the total sample size of the dataset is smaller. We compared the results of all algorithms on the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:52:12 UTC from IEEE Xplore. Restrictions apply.

TABLE XII

CLASSIFICATION RESULTS (MEAN ± STD. IN %) ON UP DATASET (FIVE LABELED SAMPLES PER CLASS)

Methods	SVM [13]	KNN [14]	SSRN [46]	SSTN [71]	3DCNN [40]	3-D DL [45]	3DCSN [68]	DCFSL [69]	HFSL [70]	ours
1	64.34±3.06	47.21±4.96	67.38±11.16	64.55±21.57	76.69±4.22	74.68±18.19	70.43±14.26	73.03±10.23	69.56±13.08	83.04±8.42
2	62.34±5.69	52.27±16.49	40.45±12.76	52.10±18.80	85.07±1.14	87.86±5.37	67.57±11.47	81.18±6.00	89.43±3.60	99.35±0.74
3	58.60±8.11	36.47±16.95	48.80±11.04	54.77±15.32	25.14±19.16	12.61±9.47	81.46±9.24	56.30±5.03	85.72±13.53	79.56±6.01
4	93.06±3.24	88.99±4.81	92.57±4.40	86.99±9.35	64.22±15.08	45.43±24.49	78.71±9.15	92.86±2.02	89.54±6.34	99.35±2.14
5	93.13±11.25	90.36±14.45	92.43±7.17	98.61±2.29	89.06±18.83	86.92±18.22	95.52±4.05	99.07±0.86	99.91±0.14	99.48±4.16
6	44.23±17.08	35.84±19.17	38.28±7.17	50.39±10.75	36.09±6.62	17.80±13.06	93.25±6.53	72.19±11.39	96.14±4.16	96.14±2.92
7	90.45±1.85	87.92±6.06	64.82±20.98	84.72±25.94	25.59±24.41	2.05±01.95	98.55±1.87	78.05±5.42	89.04±13.24	78.34±9.18
8	62.74±9.77	68.87±13.74	59.85±11.92	65.53±21.67	68.00±15.05	62.02±26.03	72.95±17.04	59.87±9.54	92.84±9.83	83.25±6.49
9	99.77±0.10	99.85±0.05	91.10±8.60	75.09±30.44	77.61±7.40	61.95±35.57	75.29±9.49	99.51±0.43	100.00±0.00	95.33±3.76
OA	65.23±3.00	56.18±5.16	53.69±4.80	60.59±9.36	70.21±3.26	65.34±2.75	74.98±5.18	77.51±3.15	87.81±2.73	**93.35±1.34**
AA	74.29±2.05	67.53±1.71	66.19±2.71	70.31±6.91	60.83±7.23	50.15±5.38	81.53±1.82	79.12±1.49	90.24±2.16	**90.42±1.29**
kappa	56.36±3.57	46.53±4.40	44.71±4.35	52.30±9.81	60.27±4.70	52.28±3.68	68.95±5.60	71.20±3.62	84.21±3.47	**91.20±1.73**

TABLE XIII

CLASSIFICATION RESULTS (MEAN ± STD. IN %) ON PC DATASET (FIVE LABELED SAMPLES PER CLASS)

Methods	SVM [13]	KNN [14]	SSRN [46]	SSTN [71]	3DCNN [40]	3-D DL [45]	3DCSN [68]	DCFSL [69]	HFSL [70]	ours
1	99.34±0.22	99.38±0.06	99.07±0.92	99.32±0.84	97.77±1.09	98.64±0.10	99.82±0.19	99.66±0.13	99.43±0.34	99.97+0.02
2	76.27±11.68	60.34±9.72	86.59±5.10	90.55±3.95	90.56±6.62	81.58±25.15	79.46±5.13	90.24±4.98	88.11±4.05	93.03±2.02
3	79.82±16.65	79.77±8.86	88.84±8.76	87.94±5.84	63.47±25.93	62.94±27.05	87.87±5.76	92.88±2.57	80.26±9.90	95.02±2.55
4	75.87±11.54	61.14±18.04	87.93±9.67	96.84±4.73	63.98±30.51	45.38±24.35	96.76±3.12	95.16±3.57	93.83±5.70	12.36±17.58
5	79.91±9.43	52.03±10.59	85.39±8.55	87.49±6.29	65.29±18.21	72.90±23.46	84.73±7.06	88.88±4.21	94.66±5.14	87.85±3.06
6	93.35±2.67	95.73±1.64	95.39±3.79	95.56±5.50	97.36±1.91	91.41±4.24	88.41±6.29	93.79±3.21	91.75±2.13	95.02±1.52
7	77.41±7.26	59.79±20.90	82.77±4.04	86.26±2.72	79.22±2.87	80.30±4.85	90.44±6.27	83.21±4.87	87.69±9.36	93.37±7.72
8	86.26±10.19	52.40±10.39	96.63±2.32	98.19±1.56	97.55±0.82	96.43±2.17	95.66±1.83	98.13±0.77	97.05±1.38	99.90±0.02
9	99.97±0.03	99.71±0.28	99.59±0.54	96.77±1.65	90.09±4.14	66.82±32.78	84.30±6.05	99.02±1.27	96.39±4.69	81.57±12.94
OA	91.24±2.83	78.43±2.27	95.70±0.73	96.82±0.45	93.48±0.79	92.30±1.28	95.12±0.66	96.85±0.40	96.33±0.58	**96.94±0.46**
AA	85.36±3.36	73.37±3.63	91.35±1.35	93.21±0.90	82.81±3.89	77.38±3.82	89.72±0.80	93.44±0.42	92.13±1.31	**84.23±2.47**
kappa	87.82±3.78	70.77±2.79	93.93±1.02	96.77±0.63	90.81±1.12	89.10±1.80	93.13±0.01	95.54±0.56	94.82±0.81	**94.86±0.65**

TABLE XIV

CLASSIFICATION RESULTS (MEAN ± STD. IN %) ON IP DATASET (FIVE LABELED SAMPLES PER CLASS)

Methods	SVM [13]	KNN [14]	SSRN [46]	SSTN [71]	3DCNN [40]	3-D DL [45]	3DCSN [68]	DCFSL [69]	HFSL [70]	ours
1	71.71±16.44	84.39±3.96	99.38±1.25	94.37±2.34	0.00±0.00	0.00±0.00	99.51±0.98	81.95±15.85	99.02±1.19	43.90±9.38
2	35.84±13.32	28.78±8.51	37.95±5.45	53.65±12.57	12.77±15.32	25.74±11.39	55.98±8.02	40.72±10.00	43.27±10.90	36.21±15.47
3	36.22±12.25	29.12±9.99	47.65±7.75	53.43±26.65	9.73±11.61	27.66±16.79	70.67±5.44	31.08±8.06	53.53±16.68	68.34±9.46
4	38.10±20.03	34.40±9.31	82.83±13.05	94.34±3.86	2.37±2.96	3.73±2.55	76.21±22.40	53.02±15.38	88.45±13.25	62.50±4.29
5	71.72±11.60	43.47±24.95	78.42±8.88	75.52±9.09	21.32±26.30	16.86±13.22	74.64±7.63	67.45±6.49	67.99±16.00	89.33±2.71
6	69.43±17.10	61.52±10.49	95.38±3.51	89.13±7.15	17.32±20.05	63.84±11.52	91.42±3.72	78.65±6.86	80.03±7.11	89.52±3.21
7	91.30±2.75	93.04±2.13	100.00±0.00	100.00±0.00	0.00±0.00	0.00±0.00	100.00±0.00	99.13±1.74	99.13±1.74	89.57±8.06
8	76.19±7.07	33.36±6.87	89.69±8.16	94.06±7.08	35.06±43.14	88.54±11.50	98.94±1.34	84.82±12.05	98.31±2.86	89.89±6.71
9	85.33±13.60	65.33±14.24	95.00±10.00	95.00±10.00	0.00±0.00	0.00±0.00	100.00±0.00	97.33±3.27	100.00±0.00	97.33±3.26
10	43.04±17.85	22.73±12.09	60.24±14.57	65.54±6.77	12.96±16.70	16.75±7.66	67.61±10.05	60.97±6.47	68.89±7.68	76.15±7.25
11	24.18±4.49	21.83±9.62	60.51±12.20	66.46±9.53	77.61±23.81	71.64±7.25	50.11±11.39	49.71±18.88	70.02±16.56	64.54±8.85
12	29.49±10.73	14.01±11.75	54.77±13.50	54.34±16.95	7.34±12.17	13.20±6.55	66.53±7.39	33.06±5.99	63.33±8.39	43.16±8.97
13	89.60±4.75	82.90±8.40	99.89±0.21	98.93±2.14	21.18±30.54	16.67±10.00	99.00±1.04	96.10±7.30	99.00±0.84	96.00±1.99
14	61.84±12.22	56.03±18.77	92.50±3.54	96.72±1.90	62.18±40.66	90.52±9.20	83.14±7.72	84.27±6.38	88.62±8.97	79.24±6.18
15	30.60±7.72	13.96±11.81	70.50±4.67	77.96±12.59	4.97±4.66	9.64±5.78	88.24±8.54	56.33±17.13	89.03±8.04	68.92±10.26
16	86.14±4.09	81.36±5.01	100.00±0.00	99.24±1.01	0.00±0.00	0.00±0.00	96.82±2.32	98.18±1.85	98.64±1.67	69.55±14.14
OA	44.25±4.91	33.83±1.44	66.81±2.29	**72.06±2.69**	35.03±6.03	46.77±2.45	69.45±3.54	57.83±5.40	70.46±3.59	70.53±1.89
AA	58.80±3.74	47.89±1.98	79.04±0.84	**81.79±2.26**	17.80±8.45	27.80±1.78	82.43±0.47	69.55±2.62	81.70±1.43	72.76±1.41
kappa	38.19±5.36	26.61±1.67	62.47±2.30	**68.21±3.14**	20.74±11.39	37.93±2.58	65.90±3.70	52.48±5.70	62.09±3.79	63.82±1.96

PC dataset. The experimental results are shown in Table XIII. Most of the algorithms in the table have high classification accuracy. Except for the KNN algorithm, the classification accuracy of all algorithms exceeds 90%, and the classification accuracy of our algorithm is up to 96.94%. It can also be seen from Fig. 13 that, except for KNN, 3-DCNN, and 3-D-DL, there are significant deviations in the prediction of class 8 (Meadows), other algorithms have high classification accuracy for each class. Our algorithm is highly close to the ground truth map, which reflects the excellent performance of our algorithm under a data-deficient scene.

Different from other datasets, the classes distribution of the IP dataset is extremely imbalanced. Among the 16 classes contained in the dataset, the number of samples of each category varies from dozens to thousands, which causes great difficulties for classification. In the small sample scenario, the impact of category imbalance is more obvious. Table XIV compares the classification results of each algorithm on the IP dataset. As can be seen from the results in the table, the classification effect of almost all algorithms on IP datasets is not satisfactory. The highest classification accuracy is 72.06%, which is obtained by the SSTN algorithm. Compared with other algorithms, the classification accuracy of our algorithm reaches 70.53%, higher than all algorithms except SSTN, but lower than HFSL with similar overall classification accuracy in average classification accuracy for each category. It can be found from Fig. 14 that 3-DCNN and 3-D DL perform the worst among all algorithms, almost failing to distinguish all classes. Experiments show that our algorithm is sensitive to class imbalance, but it still has better robustness compared with most other existing algorithms. The class imbalance is an obstacle to the hyperspectral classification of small samples,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:52:12 UTC from IEEE Xplore. Restrictions apply.

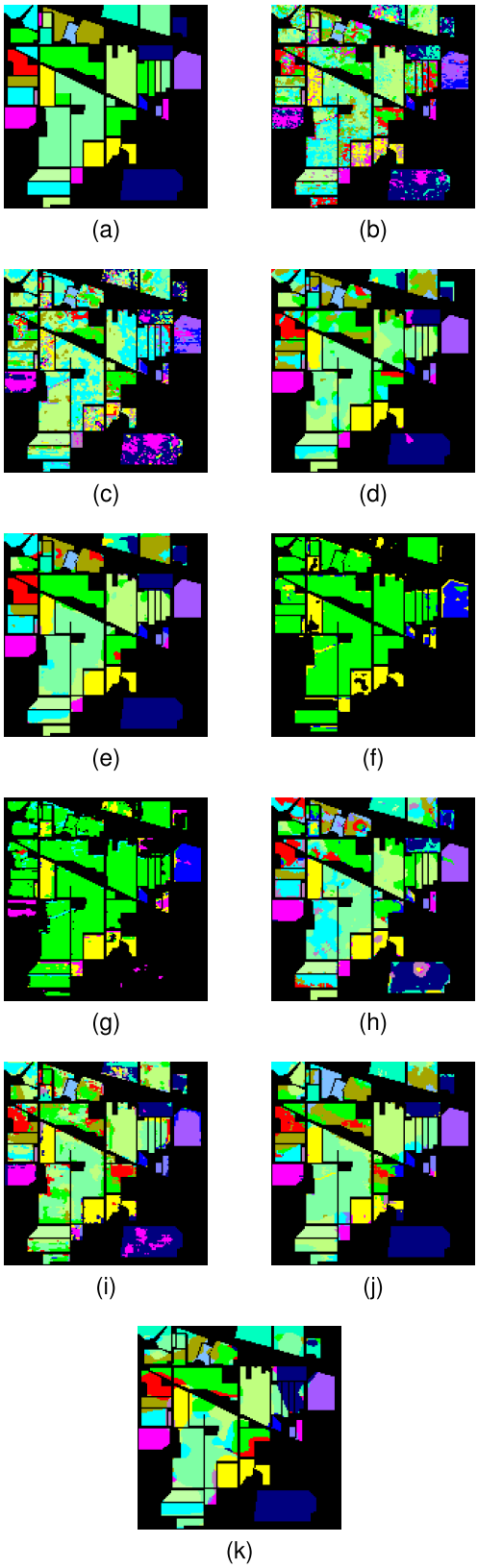


Fig. 14. IP. (a) GroundTruth. (b)–(k) Classification maps for different methods. (b) SVM. (c) KNN. (d) SSRN. (e) SSTN. (f) 3-DCNN. (g) 3-D-DL. (h) 3-DCSN. (i) DCFSL. (j) HFSL. (k) Ours.

and the sample selection strategy needs to be further optimized to reduce the influence of classes distribution on classification as much as possible.

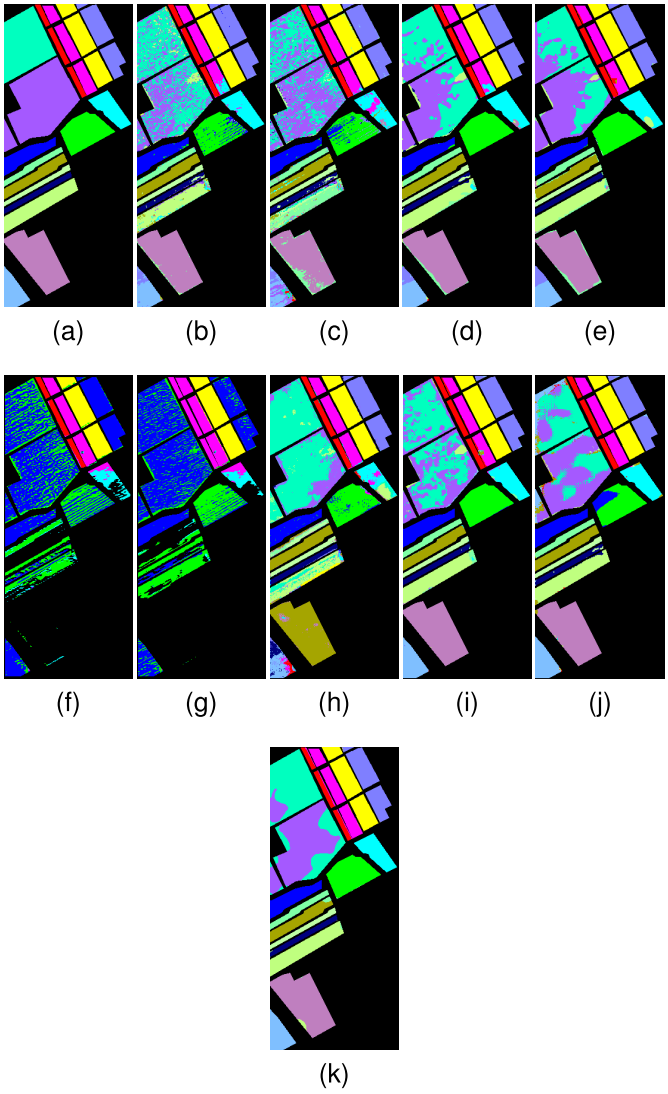


Fig. 15. Salinas. (a) GroundTruth. (b)–(k) Classification maps for different methods. (b) SVM. (c) KNN. (d) SSRN. (e) SSTN. (f) 3-DCNN. (g) 3-D-DL. (h) 3-DCSN. (i) DCFSL. (j) HFSL. (k) Ours.

To prove the generalization of the algorithm, we conducted experiments on another 16-class Salinas dataset. Unlike the UP, PC, and IP datasets, the classes of the Salinas dataset are crop cover. Table XV shows the classification performance of the different methods on the Salinas dataset. The classification map of different algorithms is shown in Fig. 15. Compared with other methods, our algorithm can achieve the highest classification accuracy of 92.70%, which further demonstrates its generalization and robustness. The average classification accuracy of our algorithm for each category reaches 96.13%, which is much higher than other algorithms. The classification accuracy of the algorithm is 100% in seven categories, including class 1, class 2, class 4, class 7, class 9, class 14, and class 16, which is significantly reflected in the classification map. The performance of traditional algorithms including SVM and KNN in small sample classification on such multicategory datasets is significantly inadequate, with classification accuracy of 81.06% and 75.53%, respectively, lower than our algorithm 11.64% and 16.97%. Worse than these two algorithms are the deep learning networks 3-DCNN,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:52:12 UTC from IEEE Xplore. Restrictions apply.

TABLE XV

CLASSIFICATION RESULTS (MEAN ± STD. IN %) ON SALINAS DATASET (FIVE LABELED SAMPLES PER CLASS)

Methods	SVM [13]	KNN [14]	SSRN [46]	SSTN [71]	3DCNN [40]	3-D DL [45]	3DCSN [68]	DCFSL [69]	HFSL [70]	ours
1	96.52±1.20	98.17±0.38	98.35±1.69	99.16±1.13	25.06±34.73	19.18±38.37	92.27±6.23	99.18±1.11	99.96±0.08	99.17±0.85
2	93.09±3.48	81.17±3.85	99.99±0.01	99.75±0.35	58.98±48.18	43.13±46.60	76.09±36.89	99.38±0.96	93.47±7.76	99.88±0.16
3	77.84±8.83	68.43±13.48	86.40±13.44	83.65±21.40	22.81±22.72	18.12±26.16	58.42±9.72	97.66±1.91	96.24±2.91	99.53±0.93
4	98.59±1.12	98.44±0.76	97.73±3.88	99.61±0.50	32.51±33.71	30.79±39.63	93.51±8.56	99.50±0.26	99.27±1.29	99.25±0.55
5	94.96±3.01	88.67±4.91	98.78±0.44	93.54±3.54	47.38±44.10	21.02±34.86	80.13±36.51	91.97±2.60	95.56±3.87	92.05±5.55
6	98.29±1.38	96.67±0.91	100.00±0.00	99.47±0.71	57.94±47.32	78.08±39.05	95.48±4.87	99.65±0.60	99.78±0.16	99.12±1.72
7	99.06±0.30	97.43±1.78	99.69±0.49	99.81±0.17	36.93±45.26	19.54±37.48	75.53±33.00	99.25±0.68	99.45±0.59	96.83±3.91
8	61.08±5.29	61.47±6.76	73.26±10.48	65.98±8.78	93.22±4.41	78.34±34.74	43.36±18.26	75.21±7.50	76.76±7.39	83.40±6.23
9	94.21±1.75	91.07±3.12	95.51±2.49	97.41±2.52	59.17±48.32	99.12±0.67	66.58±33.05	99.62±0.52	98.88±0.71	99.98±0.03
10	64.44±15.15	39.03±18.26	87.45±4.81	89.42±4.87	14.86±26.86	44.99±24.29	55.62±12.64	87.99±1.38	88.40±7.62	91.74±8.75
11	84.65±7.45	74.34±5.24	90.80±7.24	94.22±4.74	15.92±31.84	1.31±2.62	51.80±23.17	99.49±0.56	98.87±0.86	98.49±2.20
12	99.04±1.14	92.42±5.19	97.70±3.60	96.99±3.62	38.40±35.89	18.38±33.73	33.69±35.84	99.69±0.37	98.35±1.99	98.19±2.12
13	98.51±0.60	98.64±0.59	98.66±1.76	99.35±0.62	36.94±45.30	51.57±35.85	76.22±20.10	99.23±0.43	98.20±1.49	97.30±2.82
14	89.16±2.03	88.64±1.86	97.40±0.93	99.69±0.35	30.92±37.81	24.26±33.58	90.44±3.31	98.07±1.12	95.89±2.88	95.53±5.79
15	58.97±14.34	56.76±11.15	71.80±8.65	77.93±19.50	8.70±7.82	19.83±35.04	79.53±15.84	75.95±7.10	76.63±10.02	84.64±6.34
16	84.86±11.65	31.99±7.72	84.18±8.32	88.31±5.94	13.22±22.85	4.08±7.50	78.01±13.50	93.32±6.99	96.87±4.71	88.58±8.81
OA	80.59±1.75	74.63±2.70	87.76±1.71	87.30±3.12	47.30±22.27	48.40±9.86	75.27±7.73	89.89±1.03	90.01±2.13	**92.62±1.41**
AA	87.08±1.19	78.96±1.75	71.80±1.05	92.77±1.91	37.06±25.67	35.73±12.74	71.67±0.06	94.70±0.47	94.54±0.87	**95.23±1.33**
kappa	78.45±1.96	71.78±3.02	84.18±1.87	85.90±3.47	36.49±30.03	40.72±12.65	64.57±0.08	88.77±1.13	88.92±2.33	**92.00±0.02**

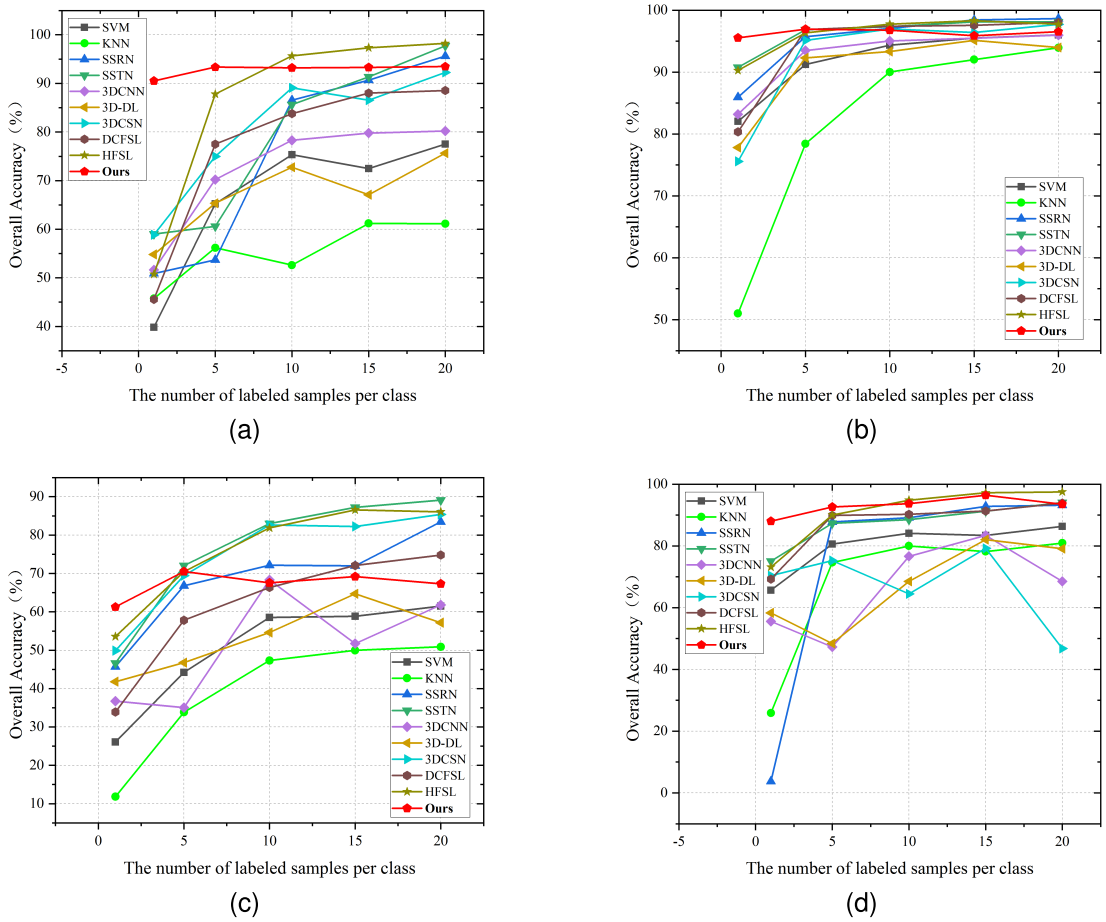


Fig. 16. Classification accuracies comparison of different methods trained with the various number of labeled samples on four HSI datasets. (a) UP. (b) PC. (c) IP. (d) Salinas.

3-D DL, and 3-DCSN constructed with 3-D convolution, with classification accuracy of, respectively, 68.61%, 64.03%, and 75.27%. In contrast, our algorithm, which also introduces 3-D convolution, is more adaptable to small sample classification because it combines the RFF kernel approximation algorithm.

In order to prove that our algorithm achieves a compromise between accuracy and efficiency in HSI classification, we calculate floating point operations (FLOPs) and the number of parameters of all comparison algorithms at different HSI datasets, as shown in Table XVI. Different from other methods, since our algorithm uses the same network architecture for different datasets, the number of FLOPs and parameters is the same for all four datasets. Other algorithms except SSRN have higher FLOPs and number of parameters on large-scale data IP and Salinas than our algorithm, especially SSRN and HFSL, whose FLOPs are even ten times higher than

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:52:12 UTC from IEEE Xplore. Restrictions apply.

TABLE XVI
FLOPs (MB) AND PARAMETERS($\times 10^5$) OF DIFFERENT HSI CLASSIFICATION ALGORITHMS

Methods		SVM	KNN	SSRN	SSTN	3DCNN	3-D DL	3DCSN	DCFSL	HFSL	ours
UP	FLOPs	-	-	107.58M	1.30M	15.13M	13.75M	47.63M	41.91M	199.49M	18.14M
	parameters	-	-	3.96	0.16	2.71	0.70	12.61	0.46	31.56	1.81
PC	FLOPs	-	-	105.42 M	1.29 M	14.98 M	13.44 M	47.63 M	41.90 M	199.47 M	18.14M
	parameters	-	-	3.89	0.16	2.68	0.67	12.61	0.46	31.53	1.81
IP	FLOPs	-	-	211.60 M	1.63 M	30.36 M	26.71 M	245.52 M	42.69 M	200.36 M	18.14M
	parameters	-	-	7.35	0.20	9.18	1.61	15.41	0.57	33.04	1.81
Salinas	FLOPs	-	-	215.93 M	1.65 M	30.98 M	27.23 M	47.63 M	42.73 M	200.40 M	18.14M
	parameters	-	-	7.49	0.21	9.36	1.61	12.65	0.58	33.10	1.81

3-DCRFF. Although SSTN algorithm has fewer complexity, it does not have advantages in practical training because it needs to use neural architecture search strategy to determine network parameters. The time complexity of 3-DCNN and 3-D-DL algorithms is slightly lower than that of 3-DCRFF on low-dimensional data. However, these two algorithms are not competitive in few-shot HSI classification accuracy. Therefore, the results demonstrates the superiority of the 3-DCRFF algorithm in terms of the time complexity.

In summary, for few-shot hyperspectral image classification, our algorithm has surpassed all current algorithms. Experiments show that our algorithm has effectiveness, robustness, and generalization, and our algorithm is the state-of-the-art few shot hyperspectral classification algorithms.

2) Impact of Different Number of Labeled Samples: In this section, we train and test the classification performance of our algorithm and the algorithms used for comparison under different input sample numbers, and analyze the influence of input sample numbers on algorithm classification performance. We randomly selected 1, 5, 10, 15, and 20 labeled training samples as the input of all algorithms. The classification results of algorithms on the four datasets are shown in Fig. 16.

VI. CONCLUSION

In this article, we propose a novel 3-DCRFF network for few-shot sample classification of hyperspectral images. We fully consider the spatial and spectral information of hyperspectral cubes and construct a new 3-DCRFF network to extract abundant spatial and spectral fusion features. For small sample scenarios, the meta-amortized variational inference is used to migrate information between different tasks in different episodes, which improves the generalization ability of the network. Experiments of different shots show that our algorithm is more suitable for the case of extremely few samples. The experimental classification accuracy of UP, PC, IP, and Salinas indicates the effectiveness of the proposed algorithm. Comparative experimental results demonstrate that the proposed algorithm is superior to other advanced algorithms for small training data.

Our algorithm increases the amount of computation after the introduction of 3-D convolution. In the future, from the perspective of practical application, we will further optimize the computational efficiency of the network.

REFERENCES

[1] X. Cao, F. Zhou, L. Xu, D. Meng, Z. Xu, and J. Paisley, "Hyperspectral image classification with Markov random fields and a convolutional neural network," *IEEE Trans. Image Process.*, vol. 27, no. 5, pp. 2354–2367, May 2018.

[2] I. Strachan, E. Pattey, and J. B. Boisvert, "Impact of nitrogen and environmental conditions on corn as detected by hyperspectral reflectance," *Remote Sens. Environ.*, vol. 80, no. 2, pp. 213–224, 2002.

[3] M. Teke, H. S. Deveci, O. Haliloglu, S. Z. Gurbuz, and U. Sakarya, "A short survey of hyperspectral remote sensing applications in agriculture," in *Proc. 6th Int. Conf. Recent Adv. Space Technol. (RAST)*, Jun. 2013, pp. 171–176.

[4] P. Kuflik and S. R. Rotman, "Band selection for gas detection in hyperspectral images," in *Proc. IEEE 27th Conv. Electr. Electron. Eng. Isr.*, Nov. 2012, pp. 1–4.

[5] S. A. El-Rahman and A. H. S. Zolait, "Hyperspectral image analysis for oil spill detection: A comparative study," *Int. J. Comput. Sci. Math.*, vol. 9, no. 2, pp. 103–121, 2018.

[6] M. Awad, "Sea water chlorophyll—A estimation using hyperspectral images and supervised artificial neural network," *Ecolog. Informat.*, vol. 24, pp. 60–68, Nov. 2014.

[7] S. Jay and M. Guillaume, "A novel maximum likelihood based method for mapping depth and water quality from hyperspectral remote-sensing data," *Remote Sens. Environ.*, vol. 147, pp. 121–132, May 2014.

[8] C. Jänicke, A. Okujeni, S. Cooper, M. Clark, P. Hostert, and S. van der Linden, "Brightness gradient-corrected hyperspectral image mosaics for fractional vegetation cover mapping in northern California," *Remote Sens. Lett.*, vol. 11, no. 1, pp. 1–10, Jan. 2020.

[9] M. Fauvel, Y. Tarabalka, J. A. Benediktsson, J. A. Chanussot, and J. C. Tilton, "Advances in spectral–spatial classification of hyperspectral images," *Proc. IEEE*, vol. 101, no. 3, pp. 652–675, Sep. 2012.

[10] J. M. Bioucas-Dias, A. Plaza, G. Camps-Valls, P. Scheunders, N. M. Nasrabadi, and J. Chanussot, "Hyperspectral remote sensing data analysis and future challenges," *IEEE Geosci. Remote Sens. Mag.*, vol. 1, no. 2, pp. 6–36, Jun. 2013.

[11] N. Audebert, B. L. Saux, and S. Lefevre, "Deep learning for classification of hyperspectral data: A comparative review," *IEEE Geosci. Remote Sens. Mag.*, vol. 7, no. 2, pp. 159–173, Jun. 2019.

[12] A. Rahimi and B. Recht, "Random features for large-scale kernel machines," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 20, 2007, pp. 1–8.

[13] J. A. Gualtieri and R. F. Cromp, "Support vector machines for hyperspectral remote sensing classification," *Proc. SPIE*, vol. 3584, pp. 221–232, Jan. 1999.

[14] B. Demir and S. Erturk, "Hyperspectral image classification using relevance vector machines," *IEEE Geosci. Remote Sens. Lett.*, vol. 4, no. 4, pp. 586–590, Oct. 2007.

[15] P. Ghamisi, J. Plaza, Y. Chen, J. Li, and A. J. Plaza, "Advanced spectral classifiers for hyperspectral images: A review," *IEEE Geosci. Remote Sens. Mag.*, vol. 5, no. 1, pp. 8–32, Mar. 2017.

[16] K. Sun *et al.*, "A robust and efficient band selection method using graph representation for hyperspectral imagery," *Int. J. Remote Sens.*, vol. 37, no. 20, pp. 4874–4889, 2016.

[17] Q. Wang, J. Lin, and Y. Yuan, "Salient band selection for hyperspectral image classification via manifold ranking," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 27, no. 6, pp. 1279–1289, Jun. 2016.

[18] S. Prasad and L. M. Bruce, "Limitations of principal components analysis for hyperspectral target recognition," *IEEE Geosci. Remote Sens. Lett.*, vol. 5, no. 4, pp. 625–629, Oct. 2008.

[19] A. Villa, J. A. Benediktsson, J. Chanussot, and C. Jutten, "Hyperspectral image classification with independent component discriminant analysis," *IEEE Trans. Geosci. Remote Sens.*, vol. 49, no. 12, pp. 4865–4876, Dec. 2011.

[20] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:52:12 UTC from IEEE Xplore. Restrictions apply.

[21] C. Szegedy *et al.*, "Going deeper with convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 1–9.

[22] G.-S. Xie *et al.*, "SRSC: Selective, robust, and supervised constrained feature representation for image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 10, pp. 4290–4302, Oct. 2020.

[23] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Region-based convolutional networks for accurate object detection and segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 38, no. 1, pp. 142–158, Jan. 2016.

[24] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2980–2988.

[25] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, 2015, pp. 1–14.

[26] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille, "DeepLab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected CRFs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 4, pp. 834–848, Apr. 2018.

[27] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 3431–3440.

[28] Z. Tian, H. Zhao, M. Shu, Z. Yang, R. Li, and J. Jia, "Prior guided feature enrichment network for few-shot segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 2, pp. 1050–1065, Feb. 2022.

[29] G.-S. Xie, X.-Y. Zhang, S. Yan, and C.-L. Liu, "Hybrid CNN and dictionary-based models for scene recognition and domain adaptation," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 27, no. 6, pp. 1263–1274, Jun. 2017.

[30] X. X. Zhu *et al.*, "Deep learning in remote sensing: A comprehensive review and list of resources," *IEEE Geosci. Remote Sens. Mag.*, vol. 5, no. 4, pp. 8–36, Dec. 2017.

[31] Y. Chen, Z. Lin, X. Zhao, G. Wang, and Y. Gu, "Deep learning-based classification of hyperspectral data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2094–2107, Jun. 2014.

[32] K. Makantasis, K. Karantzalos, A. Doulamis, and N. Doulamis, "Deep supervised learning for hyperspectral data classification through convolutional neural networks," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Jul. 2015, pp. 4959–4962.

[33] L. Mou, P. Ghamisi, and X. X. Zhu, "Deep recurrent neural networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 7, pp. 3639–3655, Apr. 2017.

[34] W. Hu, Y. Huang, L. Wei, F. Zhang, and H. Li, "Deep convolutional neural networks for hyperspectral image classification," *J. Sensors*, vol. 2015, pp. 1–12, Jan. 2015.

[35] J. Yue, W. Zhao, S. Mao, and H. Liu, "Spectral–spatial classification of hyperspectral images using deep convolutional neural networks," *Remote Sens. Lett.*, vol. 6, no. 6, pp. 468–477, Jun. 2015.

[36] J. Yue, S. Mao, and M. Li, "A deep learning framework for hyperspectral image classification using spatial pyramid pooling," *Remote Sens. Lett.*, vol. 7, no. 9, pp. 875–884, 2016.

[37] W. Zhao and S. Du, "Learning multiscale and deep representations for classifying remotely sensed imagery," *ISPRS J. Photogramm. Remote Sens.*, vol. 113, pp. 155–165, Mar. 2016.

[38] B. Liu, X. Yu, A. Yu, and G. Wan, "Deep convolutional recurrent neural network with transfer learning for hyperspectral image classification," *J. Appl. Remote Sens.*, vol. 12, no. 2, Jun. 2018, Art. no. 026028.

[39] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep feature extraction and classification of hyperspectral images based on convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 6232–6251, Jul. 2016.

[40] Y. Li, H. Zhang, and Q. Shen, "Spectral–spatial classification of hyperspectral imagery with 3D convolutional neural network," *Remote Sens.*, vol. 9, no. 1, p. 67, 2017.

[41] M. E. Paoletti, J. M. Haut, J. Plaza, and A. Plaza, "A new deep convolutional neural network for fast hyperspectral image classification," *ISPRS J. Photogram. Remote Sens.*, vol. 145, pp. 120–147, Nov. 2018.

[42] A. Plaza *et al.*, "Recent advances in techniques for hyperspectral image processing," *Remote Sens. Environ.*, vol. 113, pp. S110–S122, Sep. 2009.

[43] Y. Qian, M. Ye, and J. Zhou, "Hyperspectral image classification based on structured sparse logistic regression and three-dimensional wavelet texture features," *IEEE Trans. Geosci. Remote Sens.*, vol. 51, no. 4, pp. 2276–2291, Apr. 2013.

[44] Y. Tarabalka, J. A. Benediktsson, and J. Chanussot, "Spectral–spatial classification of hyperspectral imagery based on partitional clustering techniques," *IEEE Trans. Geosci. Remote Sens.*, vol. 47, no. 8, pp. 2973–2987, Aug. 2009.

[45] A. B. Hamida, A. Benoit, P. Lambert, and C. B. Amar, "3-D deep learning approach for remote sensing image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 8, pp. 4420–4434, Aug. 2018.

[46] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral–spatial residual network for hyperspectral image classification: A 3-D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Oct. 2017.

[47] M. Sugiyama, S. Nakajima, H. Kashima, P. Buenau, and M. Kawanabe, "Direct importance estimation with model selection and its application to covariate shift adaptation," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 20, 2007, pp. 1433–1440.

[48] A. Argyriou, T. Evgeniou, and M. Pontil, "Multi-task feature learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 19, 2006, pp. 1–23.

[49] A. Schwaighofer, V. Tresp, and K. Yu, "Learning Gaussian process kernels via hierarchical Bayes," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 17, 2004.

[50] E. V. Bonilla, K. Chai, and C. Williams, "Multi-task Gaussian process prediction," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 20, 2007.

[51] L. Mihalkova, T. Huynh, and R. J. Mooney, "Mapping and revising Markov logic networks for transfer learning," in *Proc. 22nd Nat. Conf. Artif. Intell.*, vol. 7, 2007, pp. 608–614.

[52] L. Mihalkova and R. J. Mooney, "Transfer learning by mapping with minimal target data," in *Proc. AAAI Workshop Transf. Learn. Complex Tasks*, 2008, pp. 1–6.

[53] J. Davis and P. Domingos, "Deep transfer via second-order Markov logic," in *Proc. 26th Annu. Int. Conf. Mach. Learn. (ICML)*, 2009, pp. 217–224.

[54] Y. Keneshloo, N. Ramakrishnan, and C. K. Reddy, "Deep transfer reinforcement learning for text summarization," in *Proc. SIAM Int. Conf. Data Mining*. Philadelphia, PA, USA: SIAM, 2019, pp. 675–683.

[55] K. Saito, K. Watanabe, Y. Ushiku, and T. Harada, "Maximum classifier discrepancy for unsupervised domain adaptation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 3723–3732.

[56] H. B. Ammar, E. Eaton, J. M. Luna, and P. Ruvolo, "Autonomous cross-domain knowledge transfer in lifelong policy gradient reinforcement learning," in *Proc. 24th Int. Joint Conf. Artif. Intell.*, 2015, pp. 3345–3351.

[57] Y. Qu, R. K. Baghbaderani, W. Li, L. Gao, Y. Zhang, and H. Qi, "Physically constrained transfer learning through shared abundance space for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 12, pp. 10455–10472, Dec. 2021.

[58] Y. Qu, R. K. Baghbaderani, and H. Qi, "Few-shot hyperspectral image classification through multitask transfer learning," in *Proc. 10th Workshop Hyperspectral Imag. Signal Process., Evol. Remote Sens. (WHISPERS)*, Sep. 2019, pp. 1–5.

[59] A. Santoro, S. Bartunov, M. Botvinick, D. Wierstra, and T. Lillicrap, "Meta-learning with memory-augmented neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1842–1850.

[60] J. Snell, K. Swersky, and R. Zemel, "Prototypical networks for few-shot learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–11.

[61] F. Sung, Y. Yang, L. Zhang, T. Xiang, P. H. S. Torr, and T. M. Hospedales, "Learning to compare: Relation network for few-shot learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 1199–1208.

[62] O. Vinyals *et al.*, "Matching networks for one shot learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 29, 2016, pp. 1–12.

[63] S. Ravi and H. Larochelle, "Optimization as a model for few-shot learning," Tech. Rep., 2016.

[64] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 1126–1135.

[65] K. Gao, B. Liu, X. Yu, J. Qin, P. Zhang, and X. Tan, "Deep relation network for hyperspectral image few-shot classification," *Remote Sens.*, vol. 12, no. 6, p. 923, Mar. 2020.

[66] H. Tang, Y. Li, X. Han, Q. Huang, and W. Xie, "A spatial–spectral prototypical network for hyperspectral remote sensing image," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 1, pp. 167–171, Jan. 2020.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:52:12 UTC from IEEE Xplore. Restrictions apply.

[67] H. Sun, X. Zheng, X. Lu, and S. Wu, "Spectral–spatial attention network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 5, pp. 3232–3245, May 2020.
[68] Z. Cao, X. Li, J. Jianfeng, and L. Zhao, "3D convolutional Siamese network for few-shot hyperspectral classification," *J. Appl. Remote Sens.*, vol. 14, no. 4, Nov. 2020, Art. no. 048504.
[69] Z. Li, M. Liu, Y. Chen, Y. Xu, W. Li, and Q. Du, "Deep cross-domain few-shot learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–18, 2022.
[70] Y. Wang *et al.*, "Heterogeneous few-shot learning for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.
[71] Z. Zhong, Y. Li, L. Ma, J. Li, and W.-S. Zheng, "Spectral–spatial transformer network for hyperspectral image classification: A factorized architecture search framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–15, 2022.
[72] B. Hariharan and R. Girshick, "Low-shot visual recognition by shrinking and hallucinating features," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 3018–3027.
[73] M. L. Stein, *Interpolation Spatial Data: Some Theory for Kriging*. Cham, Switzerland: Springer, 1999.
[74] S. Zhang *et al.*, "Stacked kernel network," 2017, *arXiv:1711.09219*.
[75] T. Wang, B. Dong, K. Zhang, J. Li, and L. Xu, "Slim-RFFNet: Slim deep convolution random Fourier feature network for image classification," *Knowl.-Based Syst.*, vol. 237, Feb. 2022, Art. no. 107878.
[76] S. Gidaris and N. Komodakis, "Dynamic few-shot visual learning without forgetting," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 4367–4375.
[77] S. X. Hu *et al.*, "Empirical Bayes transductive meta-learning with synthetic gradients," 2020, *arXiv:2004.12696*.
[78] D. P. Kingma, T. Salimans, and M. Welling, "Variational dropout and the local reparameterization trick," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, 2015, pp. 1–10.
[79] J. Xie, F. Liu, K. Wang, and X. Huang, "Deep kernel learning via random Fourier features," 2019, *arXiv:1910.02660*.
[80] K. Cutajar, E. V. Bonilla, P. Michiardi, and M. Filippone, "Random feature expansions for deep Gaussian processes," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 884–893.



Tingting Wang received the B.S. degree in information engineering from Jilin University, Changchun, China, in 2016, and the M.S. degree in electronic and communications engineering from Harbin Engineering University, Harbin, China, in 2018. She is currently pursuing the Ph.D. degree with the School of Cyberspace Science, Harbin Institute of Technology.

Her research interests include deep kernel learning as well as their applications to computer vision and pattern recognition, such as classification of hyperspectral images.



Huanyu Liu received the Ph.D. degree from the School of Electronic and Information Engineering, Harbin Institute of Technology, in 2021.

He is currently a Lecturer with the School of Cyberspace Science, Harbin Institute of Technology. His research interests include image super-resolution and object recognition.



Junbao Li received the Ph.D. degree from the Harbin Institute of Technology, Harbin, China, in 2008.

He is currently a Professor with the School of Cyberspace Science, Harbin Institute of Technology. His research interests are image processing and pattern recognition. He is the reviewer of many fund projects such as national natural fund, natural science fund of Heilongjiang province, natural science fund of Hebei province, and the Deputy Editor of four international journals. He has been selected into the 985 Young Scholars Support Program of Harbin Institute of Technology in 2010. He was selected as one of the first team members of the "leading wild goose" program in Heilongjiang province in 2019.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 23:52:12 UTC from IEEE Xplore. Restrictions apply.